

CẨM NANG HỆ ĐIỀU HÀNH

IT007 - UIT

Từ trực giác → bản chất → thuật toán → bài tập → Lab Linux → luyện thi

MỤC LỤC

Số trang được tạo tự động từ bản PDF hai lượt. Nhấn vào mục để chuyển đến chương tương ứng trong HTML.

Phần 0: Cách học IT007 và nền tảng C/Linux	3
Chương 1: Tổng quan về Hệ điều hành	5
Chương 2: Cấu trúc Hệ điều hành	10
Chương 3: Quản lý tiến trình	14
Chương 4: Định thời CPU	21
Giữa kỳ: Master Review và đề thi thử	26
Chương 5: Đồng bộ tiến trình	30
Chương 6: Deadlock (Bế tắc)	36
Chương 7: Quản lý bộ nhớ	40
Chương 8: Bộ nhớ ảo	44
Cuối kỳ: Master Review và 02 đề thi mô phỏng	48
Phụ lục: Linux Survival Kit và mẫu mã thực hành chọn lọc	54

PHẦN 00 — ĐỊNH HƯỚNG & NỀN TẢNG

CÁCH HỌC IT007 VÀ NỀN TẢNG CẦN BIẾT BẮT BUỘC

Biên soạn: Võ Trọng Phúc — Dành cho sinh viên UIT chinh phục điểm A/A+

Mục Tiêu Lộ Trình Phần 0

- Nắm vững tư duy học Hệ điều hành từ Trực giác → Bản chất → Thuật toán → Bài tập → Lab Linux → Bẫy đề thi UIT.
- Trang bị nền tảng C/POSIX nâng cao: Con trỏ, Bộ nhớ động, System Call, Standard I/O vs System I/O.
- Kỹ năng đọc đề thi, phân bổ thời gian và chiến thuật trình bày bài tự luận điểm tuyệt đối.

1. TRIẾT LÝ SƠ PHẠM 11 BƯỚC – HỌC HIỂU BẢN CHẤT

Môn Hệ điều hành (IT007) tại UIT nổi tiếng là một môn học vừa mang tính lý thuyết hệ thống vừa có lượng bài tập tính toán và thực hành Linux tương đối lớn. Rất nhiều sinh viên gặp thất bại vì phương pháp "học vẹt" slide hoặc học thuộc đáp án đề cũ mà không hiểu nguyên lý hoạt động của Kernel.

PHƯƠNG PHÁP CHINH PHỤC A/A+ IT007

Mỗi khái niệm trong cuốn cẩm nang này sẽ được dẫn dắt theo đúng 11 bước sơ phạm:

1. **Vì sao cần nó?** — Đặt ra bài toán thực tế máy tính gặp phải.
2. **Trực giác** — Giải thích bằng hình ảnh đời thường gần gũi.
3. **Định nghĩa chuẩn** — Thuật ngữ khoa học máy tính & chuẩn UIT.
4. **Nó hoạt động như thế nào?** — Từng bước chi tiết (Step-by-step).
5. **Hình dung trong máy tính** — Vị trí CPU, RAM, Kernel, PCB, Queue.
6. **Ví dụ nhỏ** — Bài toán cơ bản minh họa nguyên lý.
7. **Ví dụ chuẩn bài thi** — Bài tập theo đúng cấu trúc đề thi UIT.
8. **Sai lầm thường gặp** — Các lỗi tư duy khiến sinh viên mất điểm.
9. **Bẫy UIT** — Chiêu trò đổi wording hoặc cài bẫy của giảng viên.
10. **Liên hệ Lab/Linux** — Ánh xạ sang mã C, POSIX System Call và Linux Shell.
11. **Quick Recall** — Tóm tắt 3–5 dòng chốt hạ kiến thức ôn nhanh trước giờ thi.

2. NỀN TẢNG C & POSIX CẦN THIẾT CHO MÔN HỌC

IT007 sử dụng ngôn ngữ C để minh họa các khái niệm tiến trình, đồng bộ và các bài Lab Linux. Dưới đây là các khái niệm bắt buộc bạn phải vững:

2.1. Con trỏ và Quản lý bộ nhớ (Pointers & Memory Allocation)

Trong C, biến con trỏ chứa địa chỉ ô nhớ. Khi lập trình tiến trình hoặc bộ nhớ chia sẻ (Shared Memory), việc hiểu đúng địa chỉ ảo (Virtual Address) và con trỏ là chìa khóa để giải quyết bài tập.

```
// Ví dụ về địa chỉ biến và con trỏ
int a = 10;
int *p = &a; // p lưu địa chỉ ảo của biến a
printf("Giá trị a = %d, Địa chỉ &a = %p\n", a, (void*)&a);
```

⚠ ĐỪNG NHẦM: Standard I/O (printf) vs System I/O (write)

Standard I/O (`printf`) sử dụng bộ đệm (Buffer) ở Userspace. Dữ liệu chưa được ghi ngay ra màn hình cho đến khi có dấu xuống dòng `\n` hoặc gọi `fflush(stdout)`.

System Call I/O (`write`) gửi trực tiếp dữ liệu xuống Kernel space mà không đi qua bộ đệm Userspace.

Hệ quả trong bài thi UIT: Nếu gọi `fork()` ngay sau `printf("Hello")` (không có `\n`), bộ đệm Userspace bị sao chép sang tiến trình con, dẫn tới chữ "Hello" bị in 2 lần!

3. CHIẾN THUẬT LÀM BÀI THI IT007 LẤY ĐIỂM TỐI ĐA

🎯 NGUYÊN TẮC TRÌNH BÀY BÀI TỰ LUẬN UIT

- **Bài tập CPU Scheduling:** Luôn vẽ sơ đồ Gantt rõ ràng các mốc thời gian (0, 2, 4, 7..). Viết rõ công thức $TAT = CT - AT$, $WT = TAT - BT$ trước khi điền số.
- **Bài tập Process Tree (`fork`):** Không chỉ ghi đáp án số lượng process! Phải vẽ cây tiến trình từ root (Parent PID) phân nhánh sang các Child PID.
- **Bài tập Banker (Deadlock):** Phải lập đủ các bảng *Allocation*, *Max*, *Need*, *Available*. Trình bày từng bước duyệt $Work = Work + Allocation_i$.
- **Bài tập Thay thế trang (Page Replacement):** Kẻ bảng đánh dấu `\*` hoặc `H/F` (Hit/Fault) tại từng bước truy xuất reference.

📌 Quick Recall Phần 0

- Luôn đặt câu hỏi "Kernel làm gì ở background?" khi đọc bất kỳ đoạn code C nào trong môn học.
- Phân biệt rạch ròi giữa Userspace (ứng dụng) và Kernel space (Hệ điều hành).
- Trình bày bài tập tự luận cẩn thận từng bước để đạt điểm tối đa A/A+.

CHƯƠNG 01 — KIẾN THỨC NỀN TẢNG

TỔNG QUAN VỀ HỆ ĐIỀU HÀNH

Trực giác → Bản chất → Cấu trúc Phần cứng → Ngắt & Chế độ kép (Dual Mode)

Mục Tiêu Chương 1 (Chuẩn UIT IT007)

- Hiểu rõ định nghĩa, vai trò và 2 góc nhìn chính của Hệ điều hành (User View vs System View).
- Nắm vững cơ chế Ngắt (Interrupt), Phân biệt Hardware Interrupt vs Trap/Software Interrupt.
- Phân biệt chính xác các hệ điều hành: Đơn chương, Đa chương (Multiprogramming), Chia sẻ thời gian (Time-sharing), Thời gian thực (Real-time).
- Bản chất của chế độ hoạt động kép Dual-mode (User Mode vs Kernel Mode) và bảo vệ tài nguyên phần cứng.
- Giải thành thạo các câu hỏi trắc nghiệm và tự luận lý thuyết thuộc đề thi UIT.

1. BẢN CHẤT VÀ VAI TRÒ CỦA HỆ ĐIỀU HÀNH

A. Vì sao cần Hệ điều hành?

Hãy hình dung bạn có một phần cứng máy tính gồm CPU, các thanh RAM, ổ cứng SSD và card màn hình. Nếu không có Hệ điều hành, mỗi lập trình viên khi viết phần mềm (ví dụ: ứng dụng gõ văn bản) sẽ phải tự viết mã điều khiển vi mạch để đọc/ghi từng sector trên ổ cứng, tự quản lý từng byte RAM và tự phát tín hiệu điện điều khiển màn hình. Điều này vừa nguy hiểm, vừa dễ gây xung đột và lãng phí công sức tái tạo lại bánh xe đạp.

B. Trực giác

Hệ điều hành giống như **Ban Quản Lý của một tòa nhà văn phòng cao tầng**:

- Các công ty thuê văn phòng chính là các **Ứng dụng (Applications)**.
- Điện, nước, thang máy, bãi đỗ xe là các **Tài nguyên phần cứng (CPU, RAM, Disk, I/O)**.
- Ban quản lý không trực tiếp tạo ra sản phẩm cho công ty, nhưng họ đứng ra phân chia phòng làm việc (RAM), điều phối thời gian dùng thang máy (CPU) và đảm bảo công ty này không tự ý xâm nhập vào văn phòng của công ty khác (Memory Protection).

C. Định nghĩa chuẩn (UIT / OS Concepts)

ĐỊNH NGHĨA HỆ ĐIỀU HÀNH

Hệ điều hành (Operating System - OS) là một chương trình phần mềm chạy trên máy tính, đóng vai trò trung gian giữa người dùng/ứng dụng và phần cứng máy tính.

Hệ điều hành được quan sát dưới 2 góc nhìn chính:

1. **Góc nhìn người dùng (User View):** Tối ưu hóa tính dễ sử dụng (Ease of use), sự thuận tiện (Convenience) và hiệu năng cá nhân.
2. **Góc nhìn hệ thống (System View):**
 - **Bộ quản lý tài nguyên (Resource Allocator):** Quản lý và phân phối CPU, bộ nhớ, không gian đĩa, thiết bị I/O cho các tiến trình một cách công bằng và hiệu quả.

- **Chương trình kiểm soát (Control Program):** Kiểm soát việc thực thi các chương trình ứng dụng để ngăn ngừa lỗi và việc sử dụng máy tính bất hợp pháp.

2. PHÂN LOẠI HỆ ĐIỀU HÀNH VÀ CÁC THÁI CỰC XỬ LÝ

Trong lịch sử phát triển và các dạng đề thi UIT, HDH được phân loại dựa trên góc độ hình thức xử lý:

⚠ ĐỪNG NHẦM: ĐA CHƯƠNG vs CHIA SẼ THỜI GIAN vs ĐA XỬ LÝ

TIÊU CHÍ	ĐA CHƯƠNG (MULTIPROGRAMMING)	CHIA SẼ THỜI GIAN (TIME-SHARING)	ĐA XỬ LÝ (MULTIPROCESSING)
Mục tiêu chính	Tối đa hóa hiệu suất CPU (CPU Utilization). Tận dụng thời gian CPU rảnh khi I/O.	Tối thiểu hóa thời gian đáp ứng (Response Time). Tạo cảm giác tương tác tức thì.	Tối đa hóa năng lực tính toán thông qua nhiều CPU/Core vật lý.
Cơ chế chuyển đổi	Chuyển sang tiến trình khác <i>chỉ khi tiến trình hiện tại phải chờ I/O</i> .	Chuyển đổi tiến trình liên tục dựa trên <i>ngắt định kỳ thời gian (Time slice / Quantum)</i> .	Thực thi song song (Parallelism) thực sự trên nhiều bộ xử lý độc lập.
Tương tác người dùng	Không tương tác trực tiếp (Batch processing).	Tương tác trực tiếp (Interactive). Người dùng nhận phản hồi dưới vài giây.	Tùy thuộc vào HDH chạy trên hệ đa xử lý.

⚡ BẮY ĐỀ THI UIT: Hệ thống Thời gian thực (Real-Time Systems)

Đề thi UIT rất hay hỏi về **Hard Real-time** vs **Soft Real-time**:

- **Hard Real-time:** Bắt buộc mọi tác vụ phải hoàn thành *chính xác trong hạn định (Deadline)*. Nếu trễ deadline cho dù chỉ 1ms, toàn bộ hệ thống bị coi là thất bại thảm hại (ví dụ: Hệ thống phanh ABS, điều khiển tên lửa, thiết bị y tế y khoa). Không sử dụng bộ nhớ ảo (Virtual Memory) hay đĩa từ phụ.
- **Soft Real-time:** Ưu tiên các tác vụ quan trọng hoàn thành trước, chấp nhận sự trễ hạn định nhỏ mà không gây sụp đổ hệ thống (ví dụ: Xem video streaming, chơi game online).

3. CẤU TRÚC HỆ THỐNG MÁY TÍNH & CƠ CHẾ NGẮT (INTERRUPT)

A. Vì sao cần Ngắt (Interrupt)?

CPU có tốc độ tính toán cực nhanh (hàng tỷ phép tính/giây), trong khi các thiết bị I/O (bàn phím, ổ đĩa, card mạng) có tốc độ rất chậm. Nếu CPU phải liên tục vòng lặp kiểm tra xem bàn phím có được nhấn hay chưa (Polling), CPU sẽ bị lãng phí 99.99% năng lực. Ngắt ra đời để thiết bị chủ động thông báo cho CPU khi có dữ liệu.

B. Trực giác

Ngắt giống như việc bạn đang đọc sách (CPU đang chạy chương trình) thì **chuông điện thoại reo (Interrupt)**:

1. Bạn ghi nhớ số trang sách đang đọc dở (Lưu Trạng thái CPU / Program Counter).
2. Bạn nhắc máy trả lời điện thoại (Chạy chương trình phục vụ ngắt - ISR).
3. Trả lời xong, bạn mở đúng trang sách cũ ra đọc tiếp (Khôi phục trạng thái và tiếp tục thực thi).

C. Hình dung Luồng xử lý Ngắt trong Máy tính

QUY TRÌNH XỬ LÝ NGẮT (INTERRUPT PROCESS FLOW)

1. Thiết bị I/O phát tín hiệu ngắt đến CPU qua dòng ngắt phần cứng (Interrupt Line).
2. CPU hoãn tác vụ hiện tại, lưu lại địa chỉ của lệnh tiếp theo (*PC*) và trạng thái thanh ghi vào Stack.
3. CPU dựa vào chỉ số ngắt tra cứu trong **Bảng vectơ ngắt (Interrupt Vector Table)** để lấy địa chỉ của **Trình phục vụ ngắt (Interrupt Service Routine - ISR)** tương ứng.
4. CPU thực thi đoạn mã ISR để xử lý sự kiện I/O.
5. Sau khi ISR hoàn thành, CPU nạp lại trạng thái cũ từ Stack và tiếp tục chạy chương trình bị ngắt.

ĐỪNG NHẦM: Hardware Interrupt vs Trap / Exception

- **Hardware Interrupt (Ngắt phần cứng):** Do thiết bị ngoại vi bất đồng bộ phát ra (ví dụ: gõ bàn phím, di chuột, đĩa hoàn tất đọc/ghi, ngắt đồng hồ Timer). Không dự đoán trước được thời điểm xảy ra.
- **Trap / Exception (Ngắt phần mềm):** Do chính CPU phát ra một cách đồng bộ khi thực thi lệnh gặp lỗi (ví dụ: chia cho 0, truy xuất bộ nhớ trái phép, Page Fault) hoặc do chương trình chủ động phát lời gọi hệ thống (System Call / `int 0x80`).

4. CHẾ ĐỘ HOẠT ĐỘNG KÉP (DUAL-MODE OPERATION) & BẢO VỆ TÀI NGUYÊN

A. Bản chất Dual-Mode

Để đảm bảo một ứng dụng bị lỗi hoặc cố tình phá hoại không thể làm sụp đổ toàn bộ hệ thống hoặc can thiệp vào dữ liệu của ứng dụng khác, phần cứng CPU cung cấp ít nhất 2 chế độ hoạt động được đánh dấu bởi **Mode Bit** trong thanh ghi trạng thái phần cứng:

- **User Mode (Mode bit = 1):** Chế độ người dùng. Chỉ thực thi được các lệnh thông thường. Không thể truy xuất trực tiếp phần cứng hoặc toàn bộ RAM.
- **Kernel Mode / Supervisor Mode (Mode bit = 0):** Chế độ đặc quyền. Thực thi được tất cả mọi lệnh phần cứng (Privileged Instructions), truy xuất toàn bộ bộ nhớ và thiết bị.

B. Chuyển đổi Mode qua Lời gọi Hệ thống (System Call)

Khi ứng dụng ở User mode muốn đọc file từ đĩa:

1. Ứng dụng phát lệnh **System Call** (bản chất là một lệnh Trap/Interrupt).
2. Phần cứng lập tức đổi **Mode bit từ 1 sang 0** (chuyển sang Kernel mode).
3. Kernel kiểm tra quyền hạn, thực thi thao tác đọc đĩa an toàn.
4. Kernel hoàn tất, đổi **Mode bit từ 0 về 1** và trả kết quả cho ứng dụng Userspace.

⚠ **ĐỪNG NHẦM: Chuyển Đổi Chế Độ (Mode Switch) vs Chuyển Ngữ Cảnh Tiến Trình (Context Switch)**

- **Chuyển đổi chế độ (Mode Switch):** Xảy ra khi tiến trình gọi System Call hoặc gặp ngắt. CPU đổi Mode bit giữa User Mode (1) và Kernel Mode (0) nhưng *vẫn đang thực thi trong ngữ cảnh của chính tiến trình đó* (không thay đổi tiến trình, không nạp lại PCB, chi phí xử lý rất nhẹ).
- **Chuyển ngữ cảnh tiến trình (Context Switch):** Xảy ra khi bộ định thời quyết định dừng tiến trình P_1 để nhường CPU cho tiến trình P_2 . HDH phải lưu toàn bộ thanh ghi và PCB của P_1 , nạp PCB của P_2 , đồng thời cập nhật lại bảng trang bộ nhớ và cache (chi phí nặng hơn rất nhiều).

⚡ **BÃY ĐỀ THI UIT: Lệnh đặc quyền (Privileged Instructions)**

Đề thi UIT rất hay bắt phân loại lệnh nào là *Lệnh đặc quyền (chỉ chạy ở Kernel Mode)* và lệnh nào là *Lệnh không đặc quyền (chạy ở User Mode)*:

Lệnh đặc quyền (Privileged): Tắt/Bật ngắt ('cli'/'sti'), Đọc/Ghi thanh ghi Base & Limit bộ nhớ, Thao tác trực tiếp với cổng I/O ('in'/'out'), Chuyển đổi Mode bit, Xóa bộ nhớ RAM, Lệnh tắt máy ('halt'). Giả sử ứng dụng User cố tình chạy lệnh đặc quyền → CPU phát ra **Trap/Exception (Illegal Instruction)** và Kernel dẹp bỏ tiến trình đó ngay lập tức!

5. BÀI TẬP ÔN LUYỆN CHUẨN ĐỀ THI UIT CHƯƠNG 1

BÀI TẬP TỰ LUẬN MẪU (ĐỀ THI IT007 UIT)

Câu hỏi: Trình bày luồng chuyển đổi chế độ (Mode Switching) khi một chương trình người dùng thực hiện đọc dữ liệu từ bàn phím bằng lời gọi hệ thống `read()`. Vẽ sơ đồ chuyển đổi giữa User Mode và Kernel Mode.

Lời giải chi tiết từng bước:

1. **Ban đầu:** Tiến trình ứng dụng đang thực thi trong `User Mode` (Mode bit = 1).
2. **Phát System Call:** Chương trình gọi hàm `read()`. Hàm thư viện phát lệnh ngắt phần mềm (Trap / `int 0x80` hoặc `syscall`).
3. **Chuyển sang Kernel Mode:** Phần cứng tự động lưu trạng thái CPU và đổi `Mode bit = 0` (Kernel Mode). Tra cứu Interrupt Vector để chuyển quyền điều khiển cho System Call Handler của Kernel.
4. **Xử lý trong Kernel:** Kernel thực thi lệnh I/O đặc quyền để đọc dữ liệu từ driver bàn phím vào bộ đệm Kernel.
5. **Khôi phục User Mode:** Sau khi hoàn tất đọc dữ liệu, Kernel chép dữ liệu vào bộ đệm của ứng dụng, đổi `Mode bit = 1` (User Mode) và trả quyền điều khiển cho lệnh tiếp theo của chương trình người dùng.

 Quick Recall Chương 1

- **Dual-mode:** User mode (bit 1 - lệnh thường) ↔ Kernel mode (bit 0 - lệnh đặc quyền).
- **Ngắt (Interrupt):** Bất đồng bộ (Phần cứng I/O) vs Đồng bộ (Trap/Exception/System Call).
- **Đa chương:** Tối ưu CPU (chuyển khi chờ I/O) vs **Chia sẻ thời gian:** Tối ưu Response time (chuyển theo Time slice).
- **Real-time:** Hard (ng nghiêm ngặt deadline) vs Soft (ưu tiên tác vụ).

CHƯƠNG 02 — KIẾN TRÚC HỆ THỐNG

CẤU TRÚC HỆ ĐIỀU HÀNH

Thành phần → Lời gọi hệ thống (System Call) → Kiến trúc Monolithic vs Microkernel → Máy ảo

Mục Tiêu Chương 2 (Chuẩn UIT IT007)

- Nắm vững 5 thành phần cốt lõi của Hệ điều hành và các dịch vụ HDH cung cấp.
- Hiểu sâu bản chất Lời gọi hệ thống (System Call), API và 3 cơ chế truyền tham số cho System Call.
- Phân biệt rạch ròi giữa System Call (Kernel space) và Library Function Call (User space).
- So sánh ưu nhược điểm các kiến trúc HDH: Đơn khối (Monolithic), Phân lớp (Layered), Vi nhân (Microkernel), Module động (Modules).
- Hiểu khái niệm Máy ảo (Virtual Machine) và Phân loại Hypervisor (Type 1 vs Type 2).

1. CÁC THÀNH PHẦN VÀ DỊCH VỤ CỦA HỆ ĐIỀU HÀNH

A. 5 Thành phần Cốt lõi của Hệ điều hành

1. **Quản lý Tiến trình (Process Management):** Tạo, hủy, tạm dừng, tiếp tục tiến trình; Đồng bộ và Giao tiếp IPC.
2. **Quản lý Bộ nhớ chính (Memory Management):** Theo dõi các vùng nhớ rảnh/bận, Cấp phát và thu hồi RAM cho các tiến trình, Quản lý bộ nhớ ảo.
3. **Quản lý Lưu trữ / Tập tin (Storage & File Management):** Tạo/xóa file và thư mục, Ánh xạ tập tin vào bộ nhớ phụ, Sao lưu dự phòng.
4. **Quản lý Hệ thống I/O (I/O System Management):** Bộ đệm (Buffering), Caching, Spooling, Cung cấp giao diện Device Driver thống nhất.
5. **Bảo vệ và An ninh (Protection & Security):** Kiểm soát việc truy xuất tài nguyên của tiến trình và người dùng.

B. Các Dịch vụ Hệ điều hành Cung cấp

HDH cung cấp môi trường thực thi thuận tiện thông qua các dịch vụ:

- **Giao diện người dùng (User Interface):** CLI (Command Line Interface), GUI (Graphical User Interface), Batch.
- **Thực thi chương trình (Program Execution):** Nạp chương trình vào RAM và chạy.
- **Thao tác I/O (I/O Operations):** Đọc/ghi thiết bị an toàn.
- **Thao tác Hệ thống tập tin (File-system Manipulation):** Đọc, ghi, tạo, xóa, tìm kiếm tập tin.
- **Truyền thông (Communications):** Trao đổi thông điệp giữa các tiến trình trên cùng máy hoặc qua mạng.
- **Phát hiện lỗi (Error Detection):** Xử lý lỗi phần cứng (RAM error, power failure) và lỗi phần mềm (overflow, illegal memory access).

2. LỜI GỌI HỆ THỐNG (SYSTEM CALLS) VÀ CƠ CHẾ TRUYỀN THAM SỐ

A. Vì sao cần System Call?

Chương trình ứng dụng ở User mode bị cấm truy xuất trực tiếp phần cứng. Do đó, khi cần thao tác với đĩa, màn hình, bàn phím hoặc tạo tiến trình mới, ứng dụng *phải đưa ra lời yêu cầu chính thức đến Kernel* thông qua Lời gọi hệ thống (System Call).

B. Phân biệt System Call vs Library Function Call

⚠ ĐỪNG NHẦM: System Call vs Library Call

TIÊU CHÍ	LỜI GỌI HỆ THỐNG (SYSTEM CALL)	HÀM THƯ VIỆN (LIBRARY CALL)
Không gian thực thi	Thực thi trong Kernel Mode (Mode bit = 0).	Thực thi trong User Mode (Mode bit = 1).
Ví dụ trong C	<code>read()</code> , <code>write()</code> , <code>fork()</code> , <code>open()</code>	<code>printf()</code> , <code>scanf()</code> , <code>strlen()</code> , <code>strcpy()</code>
Chi phí thực thi	Chi phí cao hơn lời gọi hàm thông thường do phải chuyển đặc quyền User mode ↔ Kernel mode và thực thi xử lý trong kernel.	Chi phí thấp (chỉ là lệnh chuyển hướng nhảy hàm <code>CALL</code> trong RAM).
Mối quan hệ	Là điểm tựa cuối cùng tiếp xúc với phần cứng.	Thường bọc (wrapper) bên ngoài một hoặc nhiều System Call (ví dụ: <code>printf</code> gọi <code>write</code>).

Ghi nhớ về chi phí: một system call thường có chuyển đặc quyền từ user mode sang kernel mode và quay lại. Điều này *không tự động đồng nghĩa* với context switch; context switch chỉ xảy ra khi kernel chuyển CPU từ luồng/tiến trình đang chạy sang một luồng/tiến trình khác, chẳng hạn khi bị chặn, hết quantum hoặc bị trung dụng.

C. 3 Cơ chế Truyền Tham Số cho System Call

Khi ứng dụng gọi System Call, nó cần truyền các tham số (ví dụ: tên file, kích thước bộ đệm) cho Kernel. Có 3 phương pháp chính:

- Truyền qua Thanh ghi (Registers):** Đưa các giá trị tham số trực tiếp vào các thanh ghi CPU (ví dụ: thanh ghi `eax`, `ebx` trên x86).
 - Ưu điểm:* Rất nhanh.
 - Nhược điểm:* Bị giới hạn bởi số lượng thanh ghi của CPU (thường không đủ nếu tham số quá nhiều).
- Truyền qua Khối bộ nhớ / Bảng (Block / Table):** Lưu tham số trong một vùng nhớ RAM, sau đó truyền *địa chỉ của vùng nhớ đó* vào một thanh ghi CPU.
 - Ưu điểm:* Không bị giới hạn số lượng và kích thước tham số. Đây là phương pháp phổ biến nhất trong Linux/Solaris.
- Truyền qua Stack (Push/Pop):** Đẩy các tham số vào Stack của chương trình, Kernel sẽ Pop tham số ra từ Stack.
 - Ưu điểm:* Đơn giản, không giới hạn số tham số.

3. CÁC KIẾN TRÚC HỆ ĐIỀU HÀNH (OS STRUCTURES)

Đề thi UIT rất hay yêu cầu so sánh ưu và nhược điểm của các mô hình cấu trúc HDH:

⚠ **ĐỪNG NHẦM: Monolithic Kernel vs Microkernel**

TIÊU CHÍ	KIẾN TRÚC ĐƠN KHỐI (MONOLITHIC KERNEL)	KIẾN TRÚC VI NHÂN (MICROKERNEL)
Tổ chức Kernel	Tất cả các dịch vụ (Process Management, Memory, File System, Network, Drivers) nằm chung trong cùng một không gian địa chỉ Kernel .	Chỉ giữ lại những dịch vụ tối thiểu trong Kernel (IPC, quản lý bộ nhớ cơ bản, CPU scheduling). Các dịch vụ khác (File System, Drivers) được đẩy ra User space dưới dạng các User Services.
Hiệu năng (Performance)	Rất nhanh vì các thành phần gọi nhau qua lời gọi hàm nội bộ, không tốn chi phí Mode switch.	Chậm hơn vì các dịch vụ phải giao tiếp với nhau thông qua cơ chế truyền thông điệp (IPC) đi qua Kernel.
Độ tin cậy & An toàn	Kém hơn: Nếu một Driver bị crash hay có lỗi bộ nhớ, toàn bộ hệ thống (Kernel Panic) sẽ bị sụp đổ.	Rất cao: Nếu một Driver ở Userspace bị lỗi crash, chỉ dịch vụ đó bị dừng và có thể khởi động lại mà không ảnh hưởng tới Kernel.
Khả năng mở rộng	Khó bảo trì và mở rộng khi codebase quá lớn.	Rất dễ mở rộng và port sang kiến trúc phần cứng mới.
Ví dụ thực tế	Linux, MS-DOS, UNIX truyền thống.	Mach, QNX, L4, Symbian OS.

Cấu trúc Module động (Loadable Kernel Modules - LKM)

Các hệ điều hành hiện đại (như Linux) kết hợp ưu điểm của cả hai: Dùng kiến trúc Monolithic để đạt hiệu năng cao, nhưng hỗ trợ **Modules động**. Các driver và tính năng mới có thể được nạp ('insmod') hoặc gỡ bỏ ('rmmod') trực tiếp vào Kernel lúc đang chạy mà không cần biên dịch lại toàn bộ Kernel hay khởi động lại máy tính.

4. KHÁI NIỆM MÁY ẢO (VIRTUAL MACHINES)

A. Bản chất Máy ảo

Máy ảo (Virtual Machine - VM) là một sự trừu tượng hóa phần cứng vật lý, tạo ra giao diện ảo giống hệt như một máy tính thực thụ. Một phần mềm gọi là **Hypervisor / Virtual Machine Monitor (VMM)** quản lý các máy ảo.

B. Phân loại Hypervisor

⚡ **BÃY ĐỀ THI UIT: Hypervisor Type 1 vs Type 2**

- Type 1 (Bare-metal Hypervisor):** Chạy *trực tiếp trên phần cứng vật lý* mà không cần HDH chủ (ví dụ: VMware ESXi, Xen, KVM, Microsoft Hyper-V). Cho hiệu năng cực cao, chuyên dùng cho Server/Cloud.
- Type 2 (Hosted Hypervisor):** Chạy như *một ứng dụng trên nền một HDH chủ (Host OS)* (ví dụ: VMware Workstation, Oracle VirtualBox). Dễ cài đặt, phù hợp cho người dùng cá nhân.

5. BÀI TẬP TỰ LUẬN MẪU CHƯƠNG 2

BÀI TẬP TỰ LUẬN MẪU (ĐỀ THI IT007 UIT)

Câu hỏi: Trình bày khái niệm Microkernel. Phân tích ưu điểm và nhược điểm của kiến trúc Microkernel so với kiến trúc Monolithic. Tại sao hầu hết các HDH thương mại hiện đại (như Linux, Windows) lại áp dụng mô hình Module hóa lai (Hybrid/Modular) thay vì Microkernel thuần túy?

Lời giải chi tiết từng bước:

1. **Định nghĩa Microkernel:** Lược bỏ tối đa các thành phần khỏi Kernel, chỉ giữ lại chức năng cốt lõi (IPC, CPU scheduling, quản lý bộ nhớ cơ bản). Các thành phần như File System, Device Driver được đưa ra Userspace thành các tiến trình phục vụ độc lập.
2. **Ưu điểm của Microkernel:**
 - *Tính tin cậy & An toàn:* Lỗi ở driver không làm sụp đổ toàn bộ Kernel (tránh Kernel Panic).
 - *Mở rộng & Dễ bảo trì:* Thêm dịch vụ mới chỉ cần tạo một ứng dụng Userspace.
3. **Nhược điểm của Microkernel:** Hiệu năng giảm do chi phí quá lớn của việc chuyển đổi ngữ cảnh (Context switch) và truyền thông điệp (IPC) giữa các dịch vụ qua lại giữa Userspace và Kernelpspace.
4. **Lý do áp dụng Hybrid/Modular Kernel:** Để dung hòa giữa **Hiệu năng cực cao** của Monolithic (chạy chung trong Kernel space) và **Tính linh hoạt** của Microkernel (cho phép nạp/gỡ các module động LKM lúc runtime mà không cần khởi động lại máy).

Quick Recall Chương 2

- **System Call:** Cổng giao tiếp duy nhất từ Userspace xuống Kernelpspace.
- **3 Cơ chế truyền tham số:** Registers (nhanh, ít) → Memory Block (phổ biến) → Stack.
- **Monolithic:** Nhanh nhưng dễ sụp đổ toàn bộ. **Microkernel:** An toàn nhưng chậm do tốn IPC.
- **Hypervisor:** Type 1 (Bare-metal / Server) vs Type 2 (Hosted / Desktop).

CHƯƠNG 03 — TRỌNG TÂM THI GIỮA KỲ

QUẢN LÝ TIẾN TRÌNH (PROCESS MANAGEMENT)

Trạng thái tiến trình → Cây tiến trình (Process Tree) → Cú pháp fork/exec/wait → Lập trình C trên Linux

Mục Tiêu Chương 3 (Trọng Tâm Giữa Kỳ UIT)

- Phân biệt rạch ròi Chương trình (Program) và Tiến trình (Process); Nắm vững cấu trúc bộ nhớ của tiến trình (Text, Data, Heap, Stack).
- Thuộc lòng sơ đồ 5 trạng thái tiến trình và giải chính xác dạng bài xác định chuỗi chuyển trạng thái.
- Vẽ thành thạo Cây tiến trình (Process Tree) và tính số lượng tiến trình sinh ra bởi vòng lặp `fork()`.
- Dự đoán chính xác kết quả in ra của các chương trình C phức tạp kết hợp `fork()`, `execvp()`, `wait()` và `printf()`.
- Hiểu bản chất Copy-on-Write, biến dùng chung và bộ đệm `printf`.

1. KHÁI NIỆM TIẾN TRÌNH VÀ CẤU TRÚC BỘ NHỚ

A. Vì sao cần Tiến trình?

Một file thực thi nằm trên ổ cứng (như `chrome.exe`) chỉ là một tập hợp các byte dữ liệu thụ động. Để chạy được nó, HDH phải nạp nó vào RAM, cấp phát CPU, tạo các vùng nhớ làm việc. Thực thể đang chạy đó được gọi là Tiến trình (Process).

ĐỪNG NHẦM: Program vs Process

- **Chương trình (Program):** Thực thể thụ động (Passive entity), lưu trên ổ đĩa (file thực thi/ELF/EXE), chưa tốn tài nguyên CPU/RAM.
- **Tiến trình (Process):** Thực thể chủ động (Active entity), đang thực thi trong RAM, có con trỏ lệnh (*PC*), thanh ghi CPU và tài nguyên hệ thống.

B. Cấu trúc Không gian Bộ nhớ của Tiến trình (Memory Layout)

Một tiến trình trong RAM được chia làm 4 vùng chính (từ địa chỉ thấp đến địa chỉ cao):

- **Text Segment (Mã lệnh):** Chứa mã máy thực thi của chương trình. Thường là Read-only để tránh bị sửa đổi.
- **Data Segment (Dữ liệu):** Chứa các biến toàn cục (Global) và biến tĩnh (Static) đã hoặc chưa khởi tạo.
- **Heap Segment (Bộ nhớ động):** Vùng nhớ cấp phát động lúc runtime (thông qua `malloc()`, `calloc()` trong C hoặc `new` trong C++). Tăng trưởng theo hướng địa chỉ từ thấp lên cao.
- **Stack Segment (Nguồn/Ngăn xếp):** Chứa biến cục bộ (Local variables), tham số hàm, địa chỉ trả về của hàm. Tăng trưởng theo hướng địa chỉ từ cao xuống thấp.

2. SƠ ĐỒ TRẠNG THÁI TIẾN TRÌNH (PROCESS STATE TRANSITION)

Một tiến trình trong suốt đời sống của mình sẽ chuyển qua 5 trạng thái cơ bản:

5 TRẠNG THÁI TIẾN TRÌNH VÀ NGUYÊN NHÂN CHUYỂN TRẠNG THÁI

1. **New (Mới tạo):** Tiến trình đang được tạo lập (chưa nạp xong vào RAM).
2. **Ready (Sẵn sàng):** Tiến trình đã ở trong RAM, đầy đủ tài nguyên, chỉ chờ được cấp CPU.
3. **Running (Đang chạy):** Các lệnh của tiến trình đang được CPU thực thi.
4. **Waiting / Blocked (Chờ):** Tiến trình tạm dừng chờ một sự kiện xảy ra (ví dụ: chờ I/O hoàn tất, chờ tín hiệu signal, chờ tiến trình con).
5. **Terminated (Kết thúc):** Tiến trình đã thực thi xong hoặc bị hủy, đang chờ HDH thu hồi tài nguyên.

⚡ BÃY ĐỀ THI UIT: Xác định Chuỗi Chuyển Trạng Thái Tiến Trình

Đề thi UIT rất hay cho một đoạn code C chứa vòng lặp có lệnh `printf()` hoặc thao tác I/O và hỏi: "Tiến trình đã trải qua những trạng thái nào? Vẽ sơ đồ chuyển trạng thái."

QUY TẮC VÀNG ĐỂ ĐẠT ĐIỂM TỐI ĐA:

- Bắt đầu luôn là **New** → **Ready** → **Running**.
- Mỗi khi gặp lệnh `printf()` hay đọc/ghi file: Theo mô hình bài tập giảng dạy đơn giản hóa IT007, lệnh `printf` được quy ước là một thao tác I/O → Tiến trình chuyển từ **Running** → **Waiting** (chờ thiết bị xuất xong) → **Ready** → **Running** để chạy lệnh tiếp theo. (Trong Linux thực tế, dữ liệu ghi vào user-space buffer trước khi gọi `write()` system call).
- Khi gặp lệnh `exit(0)` hoặc kết thúc hàm `main`: Chuyển thẳng từ **Running** → **Terminated** (KHÔNG đi qua trạng thái Waiting vì không có thao tác I/O wait).

3. KHỐI QUẢN LÝ TIẾN TRÌNH (PCB), CONTEXT SWITCH VÀ TIỂU TRÌNH (THREAD)

A. Khối quản lý tiến trình (Process Control Block - PCB)

Mỗi tiến trình được đại diện trong HDH bởi một cấu trúc dữ liệu C gọi là PCB. PCB chứa toàn bộ thông tin về tiến trình:

- **Process ID (PID):** Định danh duy nhất của tiến trình.
- **Process State:** Trạng thái hiện tại (Ready, Running, Waiting...).
- **Program Counter (PC):** Địa chỉ lệnh tiếp theo sẽ thực thi.
- **CPU Registers:** Giá trị các thanh ghi CPU (Accumulators, Index registers, Stack pointer).
- **CPU Scheduling Info:** Độ ưu tiên, con trỏ đến hàng đợi scheduler.
- **Memory-management Info:** Bảng trang (Page table), Base/Limit registers.
- **I/O Status Info:** Danh sách các file đang mở (File descriptors).

B. Chuyển ngữ cảnh (Context Switch)

Khi CPU chuyển từ thực thi tiến trình P_0 sang tiến trình P_1 :

1. HDH lưu toàn bộ trạng thái của P_0 vào PCB_0 .
2. HDH nạp lại trạng thái đã lưu của P_1 từ PCB_1 .
3. CPU bắt đầu thực thi lệnh của P_1 tại vị trí PC mới nạp.

Phí tổn Context Switch: Thời gian chuyển ngữ cảnh hoàn toàn là Pure Overhead (phí tổn vô ích) vì CPU không thực thi được công việc có ích nào của người dùng trong lúc switch.

C. Khái niệm Tiểu Trình (Thread) & So sánh Tiến trình vs Tiểu trình

Tiểu trình (Thread) là đơn vị sử dụng CPU cơ bản nhất bên trong một tiến trình (còn gọi là Tiến trình nhẹ - Lightweight Process).

Một tiến trình có thể chứa một hoặc nhiều luồng tiểu trình chạy song song với nhau. Tất cả các Thread trong cùng một tiến trình **DÙNG CHUNG** không gian bộ nhớ địa chỉ (Code Text, Global Data, Heap) và các tài nguyên HDH (File Descriptors, Signals). Tuy nhiên, mỗi Thread có **VÙNG NHỚ TƯ RIÊNG** gồm: Thanh ghi CPU, Con trỏ lệnh (PC) và Ngăn xếp riêng (Private Stack).

⚠ ĐỪNG NHẦM: Tiến Trình (Process) vs Tiểu Trình (Thread)

TIÊU CHÍ	TIẾN TRÌNH (PROCESS)	TIỂU TRÌNH (THREAD)
Không gian bộ nhớ	Mỗi tiến trình có không gian bộ nhớ <i>độc lập hoàn toàn</i> .	Các thread trong cùng tiến trình <i>dùng chung</i> Code, Data, Heap.
Tạo lập & Chuyển ngữ cảnh	Nặng nề, chi phí cao (tốn thời gian tạo PCB, cấp Page table).	Nhanh chóng, chi phí cực thấp (chỉ cần tạo Stack và Registers).
Giao tiếp (IPC)	Phải qua các cơ chế IPC (Shared Memory, Pipe, Message Queue).	Giao tiếp trực tiếp bằng cách đọc/ghi các biến toàn cục (Shared Data).
Mức độ ảnh hưởng lỗi	Một tiến trình bị crash không làm ảnh hưởng tiến trình khác.	Một thread bị lỗi (Segment Fault) có thể làm sụp đổ toàn bộ tiến trình.

4. QUẢN LÝ TIẾN TRÌNH TRÊN LINUX: CÁC SYSTEM CALL `fork()`, `exec()`, `wait()`

Đây là phần kiến thức quan trọng nhất của Chương 3 trong các đề thi và bài thực hành Lab UIT.

A. System call `fork()`

`pid_t fork(void);` tạo một tiến trình con hoàn toàn mới bằng cách sao chép tiến trình cha.

⚠ ĐỪNG NHẦM: Giá trị trả về của `fork()`

- Trong **Tiến trình Cha (Parent)**: `fork()` trả về **PID của tiến trình con mới sinh** (> 0).
- Trong **Tiến trình Con (Child)**: `fork()` trả về **giá trị 0**.
- Nếu thất bại (không đủ bộ nhớ): `fork()` trả về -1 .

B. Thuật toán tính số Tiến trình sinh ra từ `fork()`

🌲 QUY TẮC VỀ CÂY TIẾN TRÌNH VÀ TÍNH SỐ PROCESS

Trường hợp 1: N lệnh `fork()` liên tiếp:

```
fork(); fork(); fork(); // N = 3
```

Tổng số tiến trình sinh ra (tính cả cha) = $2^N = 2^3 = 8$ tiến trình (gồm 1 cha và 7 con).

Trường hợp 2: `fork()` trong vòng lặp `for`:

```
for (int i = 0; i < 4; i++) {
    fork();
}
```

Số tiến trình sinh ra = $2^4 = 16$ tiến trình (gồm 1 cha và 15 con).

⚡ BẮT CÔNG THỨC 2^N (CỰC KỲ QUAN TRỌNG)

Công thức 2^N **CHỈ ÁP DỤNG KHÔNG ĐIỀU KIỆN** khi tất cả N lệnh `fork()` đều được thực thi bởi MỌI tiến trình (tức không nằm trong nhánh phân chia `if (pid == 0)` hay `if (pid > 0)`).

Nếu `fork()` nằm trong nhánh điều kiện rẽ nhánh, **BẮT BUỘC PHẢI VẼ CÂY TIẾN TRÌNH** từng bước chứ KHÔNG ĐƯỢC áp dụng trực tiếp công thức 2^N !

C. Bẫy Bộ đệm `printf` và `execvp()` trong Đề thi

⚡ BÃY 1: `printf` không có `\n` trước `fork()`

```
printf("Hello"); // Không có \n => Chuỗi nằm lại trong buffer
fork();
```

Vì `printf` chưa flush, khi `fork()` xảy ra, bộ đệm Userspace bị chép sang tiến trình con. Khi cả 2 tiến trình kết thúc, chuỗi "Hello" sẽ bị in ra **2 LẦN**!

⚡ BÃY 2: Các lệnh đăng sau `execvp()`

```
if (fork() == 0) {
    printf("Child executing ls\n");
    execlp("/bin/ls", "ls", NULL);
    printf("This line will NEVER print!\n"); // Lệnh này KHÔNG BAO GIỜ CHẠY
}
```

Nếu `execvp` nạp chương trình thành công, nó GHI ĐỀ toàn bộ bộ nhớ tiến trình con. Các dòng lệnh nằm sau `execvp` sẽ bị xóa sổ và không bao giờ chạy!

5. BÀI TẬP WORKED EXAMPLE CHUẨN ĐỀ THI UIT

WORKED EXAMPLE 1: VẼ CÂY TIẾN TRÌNH VÀ DỰ ĐOÁN OUTPUT

Cho đoạn mã C sau:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    printf("Start\n");
    pid = fork();
    if (pid > 0) {
        fork();
        printf("Parent\n");
    } else if (pid == 0) {
        printf("Child\n");
    }
    return 0;
}
```

Hỏi: Khi chạy chương trình trên, có bao nhiêu tiến trình được tạo ra? Vẽ cây tiến trình và liệt kê tất cả các chuỗi chữ có thể được in ra màn hình?

Lời giải chi tiết từng bước:

1. Phân tích dòng 1: `printf("Start\n");`

Do có `\n`, chữ "Start" được in ra ngay 1 lần bởi Tiến trình gốc P_0 .

2. Lần `fork()` thứ nhất (dòng 8):

Tiến trình P_0 sinh ra con P_1 .

- Ở P_0 (Cha): `pid > 0` → Nhảy vào nhánh `if (pid > 0)`.
- Ở P_1 (Con): `pid == 0` → Nhảy vào nhánh `else if (pid == 0)`. P_1 in ra "Child" và kết thúc.

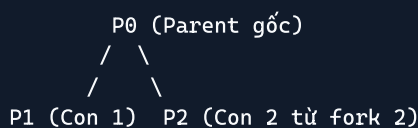
3. Lần `fork()` thứ hai (dòng 10 trong nhánh Cha):

Tiến trình cha P_0 thực thi `fork()` thứ hai → sinh thêm tiến trình con P_2 .

Cả P_0 và P_2 đều tiếp tục chạy lệnh `printf("Parent\n");`.

4. Kết luận Số lượng Tiến trình: Tổng cộng có **3 tiến trình** (P_0 gốc, P_1 con từ fork 1, P_2 con từ fork 2).

5. Cây Tiến trình (Process Tree):



6. Kết quả in ra màn hình:

- "Start": 1 lần (in ra trước khi fork).
- "Child": 1 lần (từ P_1).
- "Parent": 2 lần (từ P_0 và P_2).

⚡ LƯU Ý VỀ THỨ TỰ IN (NONDETERMINISTIC OUTPUT)

Do tiến trình cha và các tiến trình con được định thời chạy độc lập (nondeterministic execution order), thứ tự xuất hiện giữa các dòng "Child" và "Parent" trên màn hình là **KHÔNG CÓ THỨ TỰ CỐ ĐỊNH DUY NHẤT**.

Các chuỗi output đều hợp lệ:

- Start -> Child -> Parent -> Parent
- Start -> Parent -> Child -> Parent
- Start -> Parent -> Parent -> Child

WORKED EXAMPLE 2: TÍNH SỐ NGUYÊN VÀ IN VÒNG LẶP FORK

Cho chương trình C:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    for (int i = 0; i < 3; i++) {
        fork();
        printf("hello\n");
    }
    return 0;
}
```

Hỏi: Chuỗi "hello" được in ra tổng cộng bao nhiêu lần? Giải thích chi tiết bằng cây tiến trình.

Lời giải chi tiết từng bước:

1. Tại $i = 0$: 1 tiến trình gốc P_0 gọi `fork()` → tạo P_1 . Cả P_0, P_1 cùng in "hello\n" (2 lần in).
2. Tại $i = 1$: 2 tiến trình (P_0, P_1) cùng gọi `fork()` → tạo P_2, P_3 (4 tiến trình). Cả 4 tiến trình cùng in "hello\n" (4 lần in).
3. Tại $i = 2$: 4 tiến trình (P_0, P_1, P_2, P_3) cùng gọi `fork()` → tạo P_4, P_5, P_6, P_7 (8 tiến trình). Cả 8 tiến trình cùng in "hello\n" (8 lần in).
4. **Tổng số lần in = $2 + 4 + 8 = 14$ lần in chữ "hello".** (Tổng số tiến trình sinh ra là $2^3 = 8$ tiến trình).

WORKED EXAMPLE 3: ĐỊA CHỈ VẢI BIẾN VÀ COPY-ON-WRITE

Cho chương trình C:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int a = 10;
int main() {
    pid_t pid = fork();
    if (pid == 0) {
        a += 5;
        printf("Child: a = %d, &a = %p\n", a, &a);
    } else {
        wait(NULL);
        a -= 2;
        printf("Parent: a = %d, &a = %p\n", a, &a);
    }
    return 0;
}
```

Hỏi: Cho biết giá trị `a` và `&a` in ra ở tiến trình cha và tiến trình con. So sánh hai địa chỉ ảo `&a`.

Lời giải chi tiết:

- Tiến trình con (Child): In ra `Child: a = 15, &a = 0x601040` (ví dụ địa chỉ).
- Tiến trình cha (Parent): In ra `Parent: a = 8, &a = 0x601040`.
- **Giải thích bản chất:** Địa chỉ ảo `&a` của hai tiến trình là **BẰNG NHAU (GIỐNG NHAU)** vì không gian địa chỉ ảo của con được nhân bản từ cha. Tuy nhiên, MMU ánh xạ `&a` của cha và con sang 2 khung trang **RAM VẬT LÝ KHÁC NHAU** (nhờ cơ chế Copy-on-Write). Do đó giá trị biến `a` hoàn toàn độc lập!

📌 Quick Recall Chương 3

- `fork()` trả về 0 trong Child, PID con trong Parent.
- N lệnh `fork()` liên tiếp sinh 2^N tiến trình (CHỈ đúng khi fork vô điều kiện).
- `execvp()` nạp lại bộ nhớ, lệnh sau `execvp` không chạy nếu thành công.
- `printf` không `\n` bị chép buffer qua tiến trình con khi `fork()`.
- Thread chung Code/Data/Heap, riêng Stack/*PC*/Registers.
- Thứ tự in giữa các tiến trình là nondeterministic.
- `exit(0)` chuyển thẳng Running → Terminated (không qua Waiting).

CHƯƠNG 04 — TRỌNG TÂM THI GIỮA KỲ

ĐỊNH THỜI CPU (CPU SCHEDULING)

Tiêu chuẩn định thời → Gantt Chart → FCFS / SJF / SRTF / RR / Priority → Lời giải từng bước điểm 10

Mục Tiêu Chương 4 (Trọng Tâm Giữa Kỳ UIT)

- Nắm vững 5 tiêu chuẩn đánh giá thuật toán định thời (CPU_{util} , $Throughput$, TAT , WT , RT).
- Phân biệt chính xác Thuật toán Độc quyền (Non-preemptive) và Khống chế / Trưng dụng (Preemptive).
- Thao kỹ năng vẽ **Sơ đồ Gantt (Gantt Chart)** chuẩn xác theo từng mốc sự kiện thời gian.
- Tính toán chính xác 100% thời gian CT , TAT , WT , RT và thời gian trung bình cho các thuật toán FCFS, SJF, SRTF, Priority, Round Robin.
- Tránh tuyệt đối các bẫy đề thi UIT liên quan đến thứ tự vào hàng đợi Ready trong Round Robin và ngắt ưu tiên.

1. CÁC TIÊU CHUẨN ĐỊNH THỜI VÀ CÔNG THỨC BẮT BUỘC

Định thời CPU là việc chọn lựa tiến trình trong hàng đợi Ready để cấp phát CPU khi CPU rảnh.

A. Phân loại Định thời: Preemptive vs Non-Preemptive**⚠ ĐỪNG NHẦM: Non-preemptive vs Preemptive**

- Độc quyền (Non-preemptive):** Khi tiến trình đã nhận CPU, nó sẽ giữ CPU cho đến khi tự nguyện giải phóng (kết thúc hoặc chuyển sang Waiting chờ I/O). Các thuật toán: FCFS, SJF Non-preemptive, Priority Non-preemptive.
- Trưng dụng / Khống chế (Preemptive):** HDH có quyền *tước CPU của tiến trình đang chạy* để cấp cho tiến trình khác có độ ưu tiên cao hơn hoặc hết thời lượng (Time quantum). Các thuật toán: SRTF, Round Robin, Preemptive Priority.

B. 5 Tiêu chuẩn Đánh giá & Bộ Công Thức Chuẩn Bài Thi UIT

Giả sử với tiến trình P_i có các mốc thời gian:

- AT (Arrival Time): Thời điểm tiến trình đi vào Ready queue.
- BT (CPU Burst Time): Thời gian tiến trình cần dùng CPU.
- CT (Completion Time): Thời điểm tiến trình hoàn tất thực thi hoàn toàn.

▢ BỘ CÔNG THỨC TÍNH THỜI GIAN (BẮT BUỘC THUỘC LÒNG)**1. Thời gian lưu lại hệ thống / Hoàn thành (Turnaround Time - TAT):**

$$TAT = CT - AT$$

2. Thời gian chờ (Waiting Time - WT):

$$WT = TAT - BT$$

3. Thời gian đáp ứng (Response Time - RT):

$$RT = \text{Thời điểm lần đầu tiên được cấp CPU} - AT$$

4. Thời gian trung bình (Average):

$$WT_{avg} = \frac{\sum WT_i}{N}, \quad TAT_{avg} = \frac{\sum TAT_i}{N}, \quad RT_{avg} = \frac{\sum RT_i}{N}$$

2. NGUYÊN LÝ CÁC THUẬT TOÁN ĐỊNH THỜI THƯỜNG GẶP

1. FCFS (First-Come, First-Served)

Tiến trình nào đến trước (AT nhỏ hơn) được phục vụ trước. Độc quyền. Nhược điểm: Hiệu ứng đoàn tàu (Convoy Effect) nếu tiến trình đến đầu tiên có BT rất lớn làm các tiến trình sau phải chờ lâu.

2. SJF (Shortest Job First - Non-preemptive)

Chọn tiến trình có CPU Burst Time (BT) ngắn nhất trong số các tiến trình đã đến Ready Queue. Nếu hai tiến trình có BT bằng nhau thì áp dụng FCFS.

3. SRTF (Shortest Remaining Time First - Preemptive SJF)

Phiên bản khống chế của SJF. Mỗi khi có tiến trình mới xuất hiện trong Ready queue, so sánh BT còn lại của tiến trình đang chạy với BT của tiến trình mới. Nếu tiến trình mới ngắn hơn, lập tức tước CPU của tiến trình đang chạy.

4. Round Robin (RR - Thời gian luân phiên)

Mỗi tiến trình được cấp một khoảng thời gian CPU nhỏ gọi là **Time Quantum (q)**. Thực thi theo nguyên tắc FIFO trong Ready queue. Nếu hết quantum q mà tiến trình chưa xong, nó bị đưa về cuối Ready queue.

BẦY THỨ TỰ HÀNG ĐỢI READY TRONG ROUND ROBIN (ĐỀ THI UIT)

Tại mốc thời gian t , nếu tiến trình P_A vừa hết time quantum q VÀ đồng thời có tiến trình mới P_B vừa tới hệ thống ($AT = t$):

QUY TẮC UIT CHUẨN: Tiến trình mới P_B sẽ được đưa vào Ready Queue **TRƯỚC**, sau đó mới đưa tiến trình P_A (vừa hết quantum) vào **SAU** P_B !

5. Priority Scheduling (Định thời theo độ ưu tiên)

QUY ƯỚC ĐỘ ƯU TIÊN TRONG ĐỀ THI UIT

Luôn đọc quy ước ngay trong đề bài: có hệ thống dùng số nhỏ là ưu tiên cao, có hệ thống dùng số lớn là ưu tiên cao. Trong các ví dụ của cảm nang này, khi đề bài nêu rõ **số nhỏ = ưu tiên cao**, Priority 1 cao hơn Priority 2.

6. HRRN (Highest Response Ratio Next - Tỷ lệ đáp ứng cao nhất)

Thuật toán độc quyền (Non-preemptive). Mỗi khi CPU rảnh, tính Tỷ lệ đáp ứng (Response Ratio - R) của tất cả các tiến trình đã đến Ready queue. Tiến trình có R lớn nhất sẽ được chọn:

$$R = \frac{WT + BT}{BT} = 1 + \frac{WT}{BT}$$

Ưu điểm: Giúp các tiến trình có BT dài không bị đói tài nguyên (Starvation) vì khi WT tăng lên, tỷ lệ R sẽ tự động tăng theo thời gian.

7. Multilevel Queue (MQ - Hàng đợi nhiều cấp)

Hàng đợi Ready được chia thành nhiều hàng đợi riêng biệt (ví dụ: Hàng đợi Hệ thống, Hàng đợi Tương tác, Hàng đợi Batch). Mỗi hàng đợi có thuật toán định thời riêng (như RR cho hàng đợi tương tác, FCFS cho batch). Cần có lịch trình giữa các hàng đợi (thường là độ ưu tiên cố định hoặc chia tỷ lệ phần trăm thời gian CPU).

8. Multilevel Feedback Queue (MLFQ - Hàng đợi phản hồi nhiều cấp)

Cho phép tiến trình **di chuyển giữa các hàng đợi** tùy thuộc vào hành vi dùng CPU. Nếu tiến trình dùng hết time quantum mà chưa xong, nó bị hạ xuống hàng đợi ưu tiên thấp hơn. Tiến trình I/O-bound hoặc chờ lâu được nâng lên hàng đợi ưu tiên cao hơn. Đây là thuật toán linh hoạt nhất và được sử dụng trong nhiều HDH thực tế.

3. BÀI TẬP WORKED EXAMPLE THI GIỮA KỲ CHUẨN DETAILED 100%

WORKED EXAMPLE: ĐỊNH THỜI ROUND ROBIN ($q = 5$) VÀ SRTF

Cho 5 tiến trình với bảng dữ liệu sau:

PROCESS	ARRIVAL TIME (AT)	CPU BURST TIME (BT)
P1	0	13
P2	4	9
P3	6	4
P4	7	20
P5	12	10

Yêu cầu: Vẽ sơ đồ Gantt và tính WT_{avg} , TAT_{avg} , RT_{avg} cho các thuật toán:

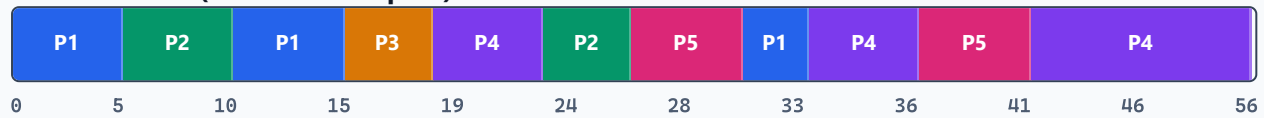
- Round Robin với $q = 5$.
- SRTF (Shortest Remaining Time First).

a) Giải Thuật Round Robin ($q = 5$):

Diễn biến Ready Queue từng mốc thời gian:

- $t = 0$: P1 đến. Chạy P1 từ $0 \rightarrow 5$. (P1 còn 8). Trong lúc này: $t = 4$ P2 đến.
- $t = 5$: Ready Queue: [P2]. Đưa P1 (vừa xong quantum) vào sau $\rightarrow$ Ready Queue: [P2, P1].
- $t = 5 \rightarrow 10$: Chạy P2 (hết quantum 5, còn 4). Trong lúc này: $t = 6$ P3 đến, $t = 7$ P4 đến. Ready Queue trước khi xếp P2 lại: [P1, P3, P4] $\rightarrow$ thêm P2 vào cuối $\rightarrow$ Ready Queue: [P1, P3, P4, P2].
- $t = 10 \rightarrow 15$: Chạy P1 (5 unit, còn 3). $t = 12$ P5 đến. Ready Queue: [P3, P4, P2, P5, P1].
- $t = 15 \rightarrow 19$: Chạy P3 (cần 4 unit $\leq 5 \rightarrow$ P3 HOÀN THÀNH tại $t = 19$, $CT_3 = 19$). Ready Queue: [P4, P2, P5, P1].
- $t = 19 \rightarrow 24$: Chạy P4 (5 unit, còn 15). Ready Queue: [P2, P5, P1, P4].
- $t = 24 \rightarrow 28$: Chạy P2 (cần 4 unit $\rightarrow$ P2 HOÀN THÀNH tại $t = 28$, $CT_2 = 28$). Ready Queue: [P5, P1, P4].

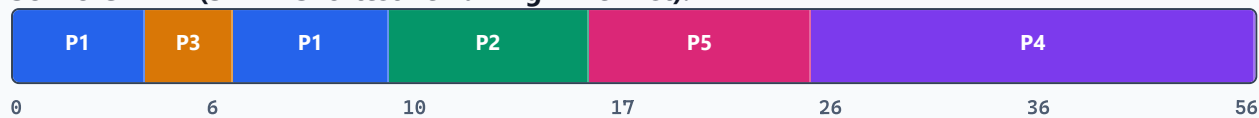
- $t = 28 \rightarrow 33$: Chạy P5 (5 unit, còn 5). Ready Queue: [P1, P4, P5].
- $t = 33 \rightarrow 36$: Chạy P1 (cần 3 unit $\rightarrow$ P1 HOÀN THÀNH tại $t = 36$, $CT_1 = 36$). Ready Queue: [P4, P5].
- $t = 36 \rightarrow 41$: Chạy P4 (5 unit, còn 10). Ready Queue: [P5, P4].
- $t = 41 \rightarrow 46$: Chạy P5 (cần 5 unit $\rightarrow$ P5 HOÀN THÀNH tại $t = 46$, $CT_5 = 46$). Ready Queue: [P4].
- $t = 46 \rightarrow 56$: Chạy P4 (còn 10 unit $\rightarrow$ P4 HOÀN THÀNH tại $t = 56$, $CT_4 = 56$).

SƠ ĐỒ GANTT (Round Robin $q = 5$):**Bảng tính chi tiết các chỉ số:**

PROCESS	AT	BT	CT	$TAT = CT - AT$	$WT = TAT - BT$	RT (LẦN ĐẦU EXEC - AT)
P1	0	13	36	36	23	$0 - 0 = 0$
P2	4	9	28	24	15	$5 - 4 = 1$
P3	6	4	19	13	9	$15 - 6 = 9$
P4	7	20	56	49	29	$19 - 7 = 12$
P5	12	10	46	34	24	$28 - 12 = 16$
TRUNG BÌNH	-	-	-	31.2	20.0	7.6

b) Giải Thuật SRTF (Shortest Remaining Time First):**Diễn biến ngắt và so sánh thời gian còn lại (Remaining BT):**

- $t = 0$: P1 đến ($BT = 13$). Chạy P1 từ $0 \rightarrow 4$. (Tại $t = 4$, P1 còn 9).
- $t = 4$: P2 đến ($BT = 9$). So sánh P1 (còn 9) và P2 (mới 9) $\rightarrow$ Hòa nhau, giữ P1 chạy tiếp $4 \rightarrow 6$. (Tại $t = 6$, P1 còn 7).
- $t = 6$: P3 đến ($BT = 4$). So sánh P3 ($BT = 4$) với P1 (còn 7) $\rightarrow$ P3 ngắn hơn! **P3 tước CPU của P1!** P3 chạy từ $6 \rightarrow 10$ (P3 HOÀN THÀNH tại $t = 10$, $CT_3 = 10$). Trong lúc này: $t = 7$ P4 đến ($BT = 20$).
- $t = 10$: P3 xong. Các tiến trình sẵn sàng: P1 (còn 7), P2 (còn 9), P4 (còn 20). P1 ngắn nhất $\rightarrow$ Chạy P1 từ $10 \rightarrow 12$. (Tại $t = 12$, P1 còn 5).
- $t = 12$: P5 đến ($BT = 10$). So sánh P1 (còn 5) với P5 (10), P2 (9), P4 (20) $\rightarrow$ P1 vẫn ngắn nhất! P1 chạy tiếp từ $12 \rightarrow 17$ (P1 HOÀN THÀNH tại $t = 17$, $CT_1 = 17$).
- $t = 17$: So sánh các tiến trình còn lại: P2 (còn 9), P5 (còn 10), P4 (còn 20). P2 ngắn nhất $\rightarrow$ Chạy P2 từ $17 \rightarrow 26$ (P2 HOÀN THÀNH tại $t = 26$, $CT_2 = 26$).
- $t = 26$: So sánh P5 (còn 10) và P4 (còn 20). P5 ngắn hơn $\rightarrow$ Chạy P5 từ $26 \rightarrow 36$ (P5 HOÀN THÀNH tại $t = 36$, $CT_5 = 36$).
- $t = 36$: Chạy P4 (20 unit) từ $36 \rightarrow 56$ (P4 HOÀN THÀNH tại $t = 56$, $CT_4 = 56$).

SƠ ĐỒ GANTT (SRTF - Shortest Remaining Time First):**Bảng tính chi tiết các chỉ số SRTF:**

PROCESS	AT	BT	CT	$TAT = CT - AT$	$WT = TAT - BT$	RT (LẦN ĐẦU EXEC - AT)
P1	0	13	17	17	4	$0 - 0 = 0$
P2	4	9	26	22	13	$17 - 4 = 13$
P3	6	4	10	4	0	$6 - 6 = 0$
P4	7	20	56	49	29	$36 - 7 = 29$
P5	12	10	36	24	14	$26 - 12 = 14$
TRUNG BÌNH	-	-	-	23.2	12.0	11.2

📌 Quick Recall Chương 4

- $TAT = CT - AT$, $WT = TAT - BT$, $RT = Exec_start - AT$.
- SJF tối ưu WT_{avg} , SRTF không chế theo BT còn lại.
- Round Robin: Tiến trình mới đến được xếp trước tiến trình vừa xong quantum!
- Độ ưu tiên: Luôn đọc kỹ quy ước của đề bài (trong đa số bài tập UIT thường quy ước số nhỏ hơn = ưu tiên cao hơn).

PHẦN REVIEW ĐẶC BIỆT — GIỮA KỲ

MIDTERM MASTER REVIEW & ĐỀ THI THỬ GIỮA KỲ CHUẨN UIT

Hệ thống hóa Ch1-Ch4 → Bẫy trắc nghiệm → 01 Đề thi mô phỏng hoàn chỉnh có đáp án

1. BÀN ĐỒ KIẾN THỨC BẮT BUỘC THI GIỮA KỲ (CH1 - CH4)

SYSTEM CHECKLIST GIỮA KỲ IT007

- **Chương 1:** Dual Mode (User mode bit 1 vs Kernel mode bit 0), Ngắt (Interrupt vs Trap/Exception), Đa chương (Multiprogramming) vs Chia sẻ thời gian (Time-sharing).
- **Chương 2:** System Call vs Library Call, 3 Cơ chế truyền tham số System Call (Registers, Memory Block, Stack), Monolithic vs Microkernel.
- **Chương 3:** 5 Trạng thái tiến trình, `fork()` return 0 ở Child / PID ở Parent, Công thức 2^N tiến trình, Bẫy buffer `printf` không `\n`, `execvp()` ghi đè bộ nhớ tiến trình.
- **Chương 4:** Công thức $TAT = CT - AT$, $WT = TAT - BT$, $RT = Exec_Start - AT$. Sơ đồ Gantt FCFS, SJF, SRTF, RR (q), Preemptive Priority (Số nhỏ = Ưu tiên cao). Thứ tự xếp hàng Ready Queue trong Round Robin.

2. ĐỀ THI MÔ PHỎNG GIỮA KỲ IT007 UIT (THỜI GIAN: 60 PHÚT)

Đề thi được biên soạn theo đúng cấu trúc đề giữa kỳ chính thức của Khoa Kỹ thuật Máy tính / Khoa CNPM - UIT.

PHẦN 1: TRẮC NGHIỆM LÝ THUYẾT (4.0 ĐIỂM)

Câu 1: Sự khác biệt cơ bản nhất giữa hệ thống Đa chương (Multiprogramming) và hệ thống Chia sẻ thời gian (Time-Sharing) là gì?

- A. Đa chương sử dụng nhiều CPU, Chia sẻ thời gian sử dụng 1 CPU.
- B. Đa chương tối ưu hiệu suất CPU (CPU_{util}), Chia sẻ thời gian tối ưu thời gian đáp ứng (RT).
- C. Đa chương hỗ trợ bộ nhớ ảo, Chia sẻ thời gian thì không.
- D. Đa chương là không chế (preemptive), Chia sẻ thời gian là không không chế (non-preemptive).

Câu 2: Lệnh nào sau đây KHÔNG PHẢI là Lệnh đặc quyền (Privileged Instruction)?

- A. Lệnh tắt/bật hệ thống ngắt phần cứng (`cli`/`sti`).
- B. Lệnh đọc thời gian hệ thống từ đồng hồ realtime.
- C. Lệnh thay đổi giá trị thanh ghi Base & Limit của RAM.
- D. Lệnh chuyển đổi Mode Bit từ User Mode sang Kernel Mode.

Câu 3: Cho đoạn mã C: `for(int i=0; i<3; i++) fork();`. Giả sử tất cả các lời gọi `fork()` đều thành công, tổng số tiến trình CON được sinh ra là:

- A. 3 tiến trình.
- B. 7 tiến trình.
- C. 8 tiến trình.
- D. 15 tiến trình.

Câu 4: Thuật toán định thời CPU nào sau đây đảm bảo KHÔNG BAO GIỜ bị ảnh hưởng bởi hiệu ứng đoàn tàu (Convoy Effect)?

- A. FCFS.
- B. Round Robin.
- C. SJF Non-preemptive.
- D. Priority Non-preemptive.

PHẦN 2: TỰ LUẬN TIẾN TRÌNH & FORK TREE (3.0 ĐIỂM)

Câu 5: Cho đoạn chương trình C sau trên Linux:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int val = 5;
    pid_t pid = fork();
    if (pid == 0) {
        val += 15;
        printf("Child: val = %d, addr = %p\n", val, &val);
    } else if (pid > 0) {
        wait(NULL);
        val -= 5;
        printf("Parent: val = %d, addr = %p\n", val, &val);
    }
    return 0;
}
```

Yêu cầu:

- a) Giả sử địa chỉ ảo của biến `val` trong tiến trình cha là `0x7ffe4a21`. Hãy xác định chính xác giá trị `val` và địa chỉ `&val` được in ra bởi tiến trình con và tiến trình cha. Giải thích bản chất cơ chế bộ nhớ.
- b) Tiến trình con đã trải qua những trạng thái nào trong suốt vòng đời của nó? Vẽ sơ đồ chuyển trạng thái.

PHẦN 3: TỰ LUẬN ĐỊNH THỜI CPU (3.0 ĐIỂM)

Câu 6: Xét tập tiến trình sau trong một hệ điều hành đơn xử lý:

PROCESS	ARRIVAL TIME (AT)	CPU BURST TIME (BT)	PRIORITY (SỐ NHỎ = ƯU TIÊN CAO)
P1	0	10	3
P2	1	3	1
P3	2	5	2
P4	4	2	4

Yêu cầu:

- a) Vẽ sơ đồ Gantt và tính WT_{avg} , TAT_{avg} cho thuật toán **Preemptive Priority Scheduling**.
- b) Vẽ sơ đồ Gantt và tính WT_{avg} , TAT_{avg} cho thuật toán **Round Robin** ($q = 3$).

3. ĐÁP ÁN VÀ LỜI GIẢI CHI TIẾT ĐỀ THI MÔ PHÒNG GIỮA KỲ

PHẦN 1: ĐÁP ÁN TRẮC NGHIỆM

- **Câu 1: B** — Đa chương mục đích tối ưu CPU_{util} (chuyển khi I/O), Chia sẻ thời gian dùng Time-slice tối ưu RT .
- **Câu 2: B** — Đọc thời gian hệ thống là lệnh không đặc quyền (User mode đọc được). Các lệnh A, C, D là Privileged.
- **Câu 3: B** — Vòng lặp $N = 3$ sinh $2^3 = 8$ tiến trình (tính cả cha). Đề hỏi số tiến trình $CON \Rightarrow 8 - 1 = 7$ con.
- **Câu 4: B** — Round Robin dùng Time quantum chia nhỏ thời gian CPU nên các tiến trình ngắn không bị kẹt sau tiến trình dài (tránh Convoy Effect).

PHẦN 2: GIẢI CHI TIẾT CÂU 5 (FORK & MEMORY)

1. Ghi kết quả in ra:

- Tiến trình con (Child): In ra `Child: val = 20, addr = 0x7ffe4a21`
- Tiến trình cha (Parent): In ra `Parent: val = 0, addr = 0x7ffe4a21`

2. Giải thích bản chất bộ nhớ:

- Khi `fork()` xảy ra, HDH sao chép không gian địa chỉ ảo của tiến trình cha sang tiến trình con (hoặc dùng Copy-on-Write).
- Do không gian địa chỉ ảo giống nhau, địa chỉ ảo `&val` ở 2 tiến trình là **BẰNG NHAU (0x7ffe4a21)**. Tuy nhiên, chúng được MMU ánh xạ sang 2 khung trang **RAM VẬT LÝ ĐỘC LẬP**.
- Do đó, biến `val` ở Child tăng thành $5 + 15 = 20$, còn `val` ở Parent sau khi `wait()` xong giảm thành $5 - 5 = 0$, hoàn toàn không ảnh hưởng lẫn nhau.

3. Chuỗi trạng thái của Tiến trình con:

`New -> Ready -> Running -> Waiting (khi printf I/O) -> Ready -> Running -> Terminated`.

PHẦN 3: GIẢI CHI TIẾT CÂU 6 (ĐỊNH THỜI CPU)

a) Preemptive Priority (Số nhỏ = Ưu tiên cao):

- $t = 0$: P1 (Prio 3) chạy.
- $t = 1$: P2 (Prio 1) đến. Vì $Prio 1 > Prio 3 \rightarrow$ P2 ngắt P1! P2 chạy từ $1 \rightarrow 4$ (P2 HOÀN THÀNH tại $t = 4$). Trong lúc này: $t = 2$ P3 (Prio 2) đến, $t = 4$ P4 (Prio 4) đến.
- $t = 4$: So sánh độ ưu tiên của P1 (Prio 3, còn $BT = 9$), P3 (Prio 2, $BT = 5$), P4 (Prio 4, $BT = 2$). P3 ưu tiên nhất $\rightarrow$ P3 chạy từ $4 \rightarrow 9$ (P3 HOÀN THÀNH tại $t = 9$).
- $t = 9$: So sánh P1 (Prio 3) và P4 (Prio 4). P1 ưu tiên hơn $\rightarrow$ P1 chạy từ $9 \rightarrow 18$ (P1 HOÀN THÀNH tại $t = 18$).
- $t = 18$: P4 chạy từ $18 \rightarrow 20$ (P4 HOÀN THÀNH tại $t = 20$).

SƠ ĐỒ GANTT (Preemptive Priority):



Tính toán:

- P1: $CT = 18, TAT = 18 - 0 = 18, WT = 18 - 10 = 8$
- P2: $CT = 4, TAT = 4 - 1 = 3, WT = 3 - 3 = 0$

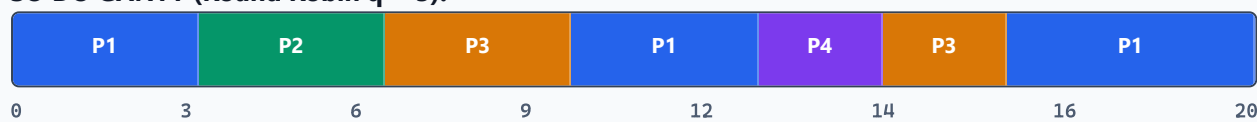
- P3: $CT = 9, TAT = 9 - 2 = 7, WT = 7 - 5 = 2$
- P4: $CT = 20, TAT = 20 - 4 = 16, WT = 16 - 2 = 14$
- $TAT_{avg} = \frac{18+3+7+16}{4} = 11.0 \text{ ms}$
- $WT_{avg} = \frac{8+0+2+14}{4} = 6.0 \text{ ms}$

b) Round Robin ($q = 3$):

Diễn biến Ready Queue từng mốc thời gian:

- $t = 0$: P1 đến ($BT = 10$). Chạy P1 từ $0 \rightarrow 3$ (còn 7). Trong lúc này: $t = 1$ P2 đến, $t = 2$ P3 đến. Ready Queue trước khi xếp P1 lại: [P2, P3] $\rightarrow$ thêm P1 vào cuối $\rightarrow$ Ready Queue: [P2, P3, P1].
- $t = 3 \rightarrow 6$: Chạy P2 (cần 3 unit $\rightarrow$ P2 HOÀN THÀNH tại $t = 6, CT_2 = 6$). Trong lúc này: $t = 4$ P4 đến. Ready Queue: [P3, P1, P4].
- $t = 6 \rightarrow 9$: Chạy P3 (3 unit, còn 2). Ready Queue: [P1, P4, P3].
- $t = 9 \rightarrow 12$: Chạy P1 (3 unit, còn 4). Ready Queue: [P4, P3, P1].
- $t = 12 \rightarrow 14$: Chạy P4 (cần 2 unit $\leq 3 \rightarrow$ P4 HOÀN THÀNH tại $t = 14, CT_4 = 14$). Ready Queue: [P3, P1].
- $t = 14 \rightarrow 16$: Chạy P3 (cần 2 unit còn lại $\rightarrow$ P3 HOÀN THÀNH tại $t = 16, CT_3 = 16$). Ready Queue: [P1].
- $t = 16 \rightarrow 19$: Chạy P1 (3 unit, còn 1). Ready Queue: [P1].
- $t = 19 \rightarrow 20$: Chạy P1 (1 unit cuối $\rightarrow$ P1 HOÀN THÀNH tại $t = 20, CT_1 = 20$).

SƠ ĐỒ GANTT (Round Robin $q = 3$):



Bảng tính chi tiết chỉ số Round Robin ($q = 3$):

PROCESS	AT	BT	CT	$TAT = CT - AT$	$WT = TAT - BT$	RT (EXEC START - AT)
P1	0	10	20	20	10	$0 - 0 = 0$
P2	1	3	6	5	2	$3 - 1 = 2$
P3	2	5	16	14	9	$6 - 2 = 4$
P4	4	2	14	10	8	$12 - 4 = 8$
TRUNG BÌNH	-	-	-	12.25 ms	7.25 ms	3.5 ms

Chúc mừng bạn đã hoàn thành Midterm Review!

Hãy soát lại các lỗi sai trong đề thi thử và ôn kỹ lại Chương 3 & Chương 4 trước khi bước sang các chương Cuối kỳ!

CHƯƠNG 05 — TRỌNG TÂM THI CUỐI KỲ

ĐỒNG BỘ TIẾN TRÌNH (PROCESS SYNCHRONIZATION)

Race Condition → Vùng tranh chấp (Critical Section) → Thuật toán Peterson → Semaphore → Các bài toán kinh điển

Mục Tiêu Chương 5 (Trọng Tâm Cuối Kỳ UIT)

- Hiểu bản chất Race Condition và tại sao truy xuất dữ liệu dùng chung bất đồng bộ lại gây ra sai sót.
- Nắm vững 3 yêu cầu bắt buộc của giải pháp Vùng tranh chấp (Mutual Exclusion, Progress, Bounded Waiting).
- Phân tích thuật toán Peterson và các chỉ thị phần cứng (TestAndSet, Swap).
- Thạo kỹ năng sử dụng Semaphore để giải các bài toán đồng bộ thứ tự thực thi (Dependency DAG) và các bài toán kinh điển (Producer-Consumer, Readers-Writers).
- Sửa lỗi Race Condition cho các chương trình thực tế chuẩn bài thi UIT.

1. NGUYÊN NHÂN RACE CONDITION VÀ BÀI TOÁN VÙNG TRANH CHẤP

A. Race Condition (Tranh chấp dữ liệu)

Khi nhiều tiến trình/thread truy xuất và thao tác trên **dữ liệu dùng chung (Shared Data)** một cách đồng thời, và kết quả cuối cùng phụ thuộc vào thứ tự thực thi ngẫu nhiên của các tiến trình, tình trạng đó gọi là Race Condition.

VÍ DỤ NGUYÊN TỬ VỀ RACE CONDITION

Giả sử 2 tiến trình P_1 và P_2 cùng thực hiện lệnh `counter++` trên biến toàn cục `counter = 5`:

Lệnh `counter++` trong mã máy thực chất gồm 3 bước:

1. `register1 = counter` (Đọc giá trị từ RAM vào thanh ghi)
2. `register1 = register1 + 1` (Tăng giá trị thanh ghi)
3. `counter = register1` (Ghi lại giá trị từ thanh ghi vào RAM)

Nếu P_1 đang chạy bước 1 và 2 thì bị ngắt, P_2 nhảy vào chạy cả 3 bước, sau đó P_1 được cấp CPU chạy nốt bước 3 → Giá trị `counter` cuối cùng sẽ là 6 thay vì 7! Dữ liệu bị sai hỏng hoàn toàn.

B. Vùng Tranh Chấp (Critical Section - CS)

Critical Section là đoạn mã nguồn của tiến trình mà trong đó tiến trình truy xuất và thay đổi dữ liệu dùng chung.

C. 3 Yêu cầu Bắt buộc cho Giải pháp CS

1. **Loại trừ tương hỗ (Mutual Exclusion):** Nếu tiến trình P_i đang thực thi trong vùng CS của nó, thì không có tiến trình P_j nào khác được phép thực thi trong vùng CS đó.

2. **Tiến triển (Progress):** Nếu không có tiến trình nào đang ở trong vùng CS và có các tiến trình muốn vào CS, thì chỉ những tiến trình không nằm trong Remainder section mới được tham gia quyết định tiến trình tiếp theo được vào CS, và sự lựa chọn này không thể kéo dài vô hạn.
3. **Chờ đợi giới hạn (Bounded Waiting):** Có một giới hạn về số lần các tiến trình khác được phép vào vùng CS của chúng sau khi một tiến trình đã đưa ra yêu cầu vào CS và trước khi yêu cầu đó được chấp nhận (Đảm bảo không tiến trình nào bị đói tài nguyên / Starvation).

2. GIẢI PHÁP ĐỒNG BỘ: THUẬT TOÁN PETERSON VÀ SEMAPHORE

A. Thuật toán Peterson cho 2 Tiến trình

Giải pháp phần mềm cổ điển cho 2 tiến trình P_0 và P_1 chia sẻ 2 biến:

```
int turn;
boolean flag[2]; // ban đầu khởi tạo false

// Cấu trúc tiến trình Pi (i = 0 hoặc 1, j = 1 - i)
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j) ; // Busy waiting

    /* CRITICAL SECTION */

    flag[i] = false;

    /* REMAINDER SECTION */
}
```

Thuật toán Peterson chứng minh thỏa mãn cả 3 điều kiện: Mutual Exclusion, Progress và Bounded Waiting.

B. Semaphore

Semaphore S là một biến nguyên tử (Atomic integer) chỉ có thể truy xuất thông qua 2 thao tác nguyên tử chuẩn:

⚙️ ĐỊNH NGHĨA NGUYÊN TỬ CỦA SEMAPHORE

Thao tác `wait(S)` (hoặc `P(S)` / `sem_wait(&S)`):

```
void wait(Semaphore S) {
    S.value--;
    if (S.value < 0) {
        // Thêm tiến trình này vào hàng đợi của Semaphore S
        block(); // Chuyển tiến trình sang trạng thái Waiting
    }
}
```

Thao tác `signal(S)` (hoặc `V(S)` / `sem_post(&S)`):

```
void signal(Semaphore S) {
    S.value++;
    if (S.value <= 0) {
        // Lấy một tiến trình P từ hàng đợi của Semaphore S
        wakeup(P); // Chuyển tiến trình P sang trạng thái Ready
    }
}
```

```
}
}
```

⚠ ĐỪNG NHẦM: Counting Semaphore vs Binary Semaphore

- **Counting Semaphore:** Giá trị có thể dao động không hạn chế. Dùng để quản lý số lượng thực thể của một tài nguyên hữu hạn.
- **Binary Semaphore:** Giá trị chỉ có thể là 0 hoặc 1. Hoạt động tương tự Mutex Lock nhưng không có tính sở hữu (Ownership). Dùng làm khóa truy xuất độc quyền hoặc đồng bộ thứ tự.

C. Đồng Bộ Phần Cứng (Hardware Instructions: TestAndSet & Swap)

Các bộ xử lý hiện đại cung cấp các chỉ thị máy nguyên tử (Atomic hardware instructions) executed không thể bị ngắt giữa chừng:

```
// 1. Chỉ thị TestAndSet nguyên tử
boolean TestAndSet(boolean *target) {
    boolean rv = *target;
    *target = true;
    return rv;
}

// Giải pháp CS dùng TestAndSet:
while (TestAndSet(&lock)) ; // Busy wait
/* CRITICAL SECTION */
lock = false;
```

D. Giám Sát Viên (Monitor) & Biến Điều Kiện (Condition Variables)

Semaphore tuy mạnh mẽ nhưng dễ gây ra lỗi lập trình (như quên `signal` hoặc đảo vị trí `wait` gây Deadlock).

Monitor (Giám sát viên) là một cấu trúc ngôn ngữ lập trình bậc cao tự động đảm bảo tính Loại trừ tương hỗ (Mutual Exclusion): tại một thời điểm chỉ có ĐÚNG MỘT tiến trình được thực thi mã bên trong Monitor.

- **Biến điều kiện (Condition Variable):** Khai báo `condition x, y;` để cho phép tiến trình tạm dừng bên trong Monitor.
- `x.wait()` : Tiến trình gọi lệnh này sẽ bị treo và giải phóng Monitor cho tiến trình khác vào.
- `x.signal()` : Đánh thức ĐÚNG MỘT tiến trình đang chờ trên `x.wait()` . Nếu không có tiến trình nào chờ, lệnh `signal` bị trôi mất (không có bộ đệm lưu trữ như Semaphore!).
- **Hoare vs Mesa Semantics:** *Signal-and-Wait (Hoare)* cho tiến trình được đánh thức chạy ngay lập tức; *Signal-and-Continue (Mesa)* cho tiến trình phát `signal` tiếp tục chạy hết Monitor rồi tiến trình được đánh thức mới chạy.

E. Hàng Rào Bộ Nhớ (Memory Barrier / Memory Fence)

Trên các kiến trúc đa luồng và đa lõi hiện đại, trình biên dịch và phần cứng CPU có thể tự động **đổi thứ tự thực thi lệnh (Instruction Reordering)** nhằm tối ưu hóa hiệu năng đường ống (Instruction Pipeline). Điều này có thể khiến các luồng khác nhìn thấy thứ tự cập nhật dữ liệu bộ nhớ không đúng.

Hàng rào bộ nhớ (Memory Barrier / Memory Fence) là một chỉ thị phần cứng buộc hệ thống phải hoàn tất tất cả các thao tác nạp (Load) và lưu (Store) trước đó vào bộ nhớ trước khi thực thi bất kỳ thao tác bộ nhớ nào sau hàng rào, bảo đảm tính nhất quán dữ liệu trong lập trình đồng thời cấp thấp.

3. CÁC BÀI TOÁN ĐỒNG BỘ KINH ĐIỂN VÀ BÀI TẬP CHUẨN THI UIT

A. 3 Bài Toán Đồng Bộ Kinh Điển

1. BÀI TOÁN NHÀ SẢN XUẤT - NGƯỜI TIÊU THỤ (PRODUCER - CONSUMER)

Bộ đệm chia sẻ gồm N phần tử. Producer tạo dữ liệu đưa vào buffer, Consumer lấy dữ liệu ra khỏi buffer.

```
semaphore mutex = 1; // Độc quyền truy xuất Buffer
semaphore empty = N; // Đếm số ô trống
semaphore full = 0; // Đếm số ô có dữ liệu

// Producer
do {
    produce_item();
    wait(empty); wait(mutex);
    append_buffer();
    signal(mutex); signal(full);
} while (true);

// Consumer
do {
    wait(full); wait(mutex);
    take_buffer();
    signal(mutex); signal(empty);
    consume_item();
} while (true);
```

2. BÀI TOÁN BẠN ĐỌC - NGƯỜI VIẾT (READERS - WRITERS)

Nhiều tiến trình chia sẻ dữ liệu. Nhiều Reader có thể đọc đồng thời; nhưng khi Writer ghi dữ liệu, phải độc quyền hoàn toàn (không ai khác được đọc/ghi).

```
semaphore rw_mutex = 1; // Độc quyền cho Writer
semaphore mutex = 1; // Bảo vệ biến read_count
int read_count = 0;

// Reader
wait(mutex);
read_count++;
if (read_count == 1) wait(rw_mutex); // Reader đầu tiên khóa Writer
signal(mutex);

/* READING DATABASE */

wait(mutex);
read_count--;
if (read_count == 0) signal(rw_mutex); // Reader cuối cùng mở khóa Writer
signal(mutex);

// Writer
wait(rw_mutex);
```

```
/* WRITING DATABASE */
signal(rw_mutex);
```

3. BÀI TOÁN BỮA ĂN CỦA CÁC TRIẾT GIA (DINING PHILOSOPHERS)

5 Triết gia ngồi quanh bàn tròn, giữa mỗi cặp triết gia có 1 chiếc đĩa (tổng 5 đĩa). Để ăn, triết gia cần 2 đĩa bên trái và bên phải.

Nguy cơ Deadlock: Nếu tất cả 5 triết gia cùng nhặt đĩa bên trái cùng lúc → Không ai nhặt được đĩa bên phải → Deadlock!

Giải pháp khắc phục: Cho phép tối đa 4 triết gia ngồi ăn cùng lúc; hoặc triết gia chỉ nhặt đĩa khi cả 2 đĩa trái/phải đều sẵn sàng (dùng Monitor / Semaphore asymmetric).

B. Bài Tập Đồng Bộ Sơ Đồ Phụ Thuộc (DAG) Chuẩn Thi UIT

WORKED EXAMPLE 1: ĐỒNG BỘ SƠ ĐỒ TIẾN TRÌNH BẰNG SEMAPHORE

Bài toán: Xét một hệ thống gồm 4 tiến trình T_1, T_2, T_3, T_4 . Quan hệ phụ thuộc được mô tả: T_1 phải kết thúc trước khi T_2 và T_3 bắt đầu. Cả T_2 và T_3 phải kết thúc trước khi T_4 bắt đầu. Hãy khai báo Semaphore và viết mã đồng bộ cho 4 tiến trình trên.

Lời giải chi tiết từng bước:

1. Phân tích quan hệ phụ thuộc (DAG):

- $T_1 \rightarrow T_2$ và $T_1 \rightarrow T_3$ (Cần tín hiệu từ T_1).
- $T_2 \rightarrow T_4$ và $T_3 \rightarrow T_4$ (T_4 cần chờ cả T_2 và T_3).

2. Khai báo và Khởi tạo Semaphore:

```
semaphore sem1 = 0; // Báo hiệu T1 đã hoàn thành
semaphore sem2 = 0; // Báo hiệu T2/T3 đã hoàn thành
```

3. Viết mã nguồn cho các tiến trình:

```
void T1() {
    // T1 thực thi công việc
    signal(sem1); // Gửi tín hiệu cho T2
    signal(sem1); // Gửi tín hiệu cho T3
}

void T2() {
    wait(sem1); // Chờ T1 xong
    // T2 thực thi công việc
    signal(sem2); // Gửi tín hiệu cho T4
}

void T3() {
    wait(sem1); // Chờ T1 xong
    // T3 thực thi công việc
    signal(sem2); // Gửi tín hiệu cho T4
}
```

```

}

void T4() {
    wait(sem2);    // Chờ T2 xong
    wait(sem2);    // Chờ T3 xong
    // T4 thực thi công việc
}

```

WORKED EXAMPLE 2: SỬA LỖI BIẾN X VƯỢT QUÁ 20

Cho đoạn code C chia sẻ biến X giữa 2 tiến trình:

```

do {
    X = X + 1;
    if (X == 20) X = 0;
} while (TRUE);

```

Hãy giải thích nguyên nhân làm $X > 20$ và dùng Semaphore để sửa lỗi.

Lời giải:

Nguyên nhân: Race condition giữa bước tăng X và bước kiểm tra `if (X == 20)`. Nếu $X = 19$, P_1 tăng X thành 20 nhưng bị ngắt trước `if`. P_2 nhảy vào tăng X thành 21. Khi P_1 chạy tiếp `if (X == 20)` → sai (vì $X = 21$) → X không về 0 và tiếp tục tăng lên 22, 23..

Khắc phục bằng Semaphore: Bọc toàn bộ đoạn tăng và kiểm tra trong Mutex Semaphore `mutex = 1`.

```

semaphore mutex = 1;
do {
    wait(mutex);
    X = X + 1;
    if (X == 20) X = 0;
    signal(mutex);
} while (TRUE);

```

Quick Recall Chương 5

- 3 Yêu cầu CS: Mutual Exclusion, Progress, Bounded Waiting.
- `wait(S)` giảm S (kết nếu $S \leq 0$), `signal(S)` tăng S (đánh thức tiến trình chờ).
- Monitor tự động khóa Mutual Exclusion, dùng `condition x; x.wait(), x.signal()`.
- Producer-Consumer: `empty=N, full=0, mutex=1`.
- Khởi tạo Semaphore = 0 cho đồng bộ thứ tự (DAG), = 1 cho độc quyền truy xuất (CS).

CHƯƠNG 06 — TRỌNG TÂM THI CUỐI KỲ

DEADLOCK (BẾ TẮC)

4 Điều kiện cần → Đồ thị RAG → Trạng thái an toàn → Thuật toán Banker từng bước

Mục Tiêu Chương 6 (Trọng Tâm Cuối Kỳ UIT)

- Nắm vững định nghĩa Deadlock và thuộc lòng 4 điều kiện cần để xảy ra bế tắc.
- Biết cách dựng và đọc Đồ thị Cấp phát Tài nguyên (Resource Allocation Graph - RAG).
- Phân biệt chính xác giữa Trạng thái An toàn (Safe State), Unsafe State và Deadlock.
- Tính toán chính xác 100% Thuật toán Banker (Dựng ma trận Need, Kiểm tra Safe Sequence, Xử lý Yêu cầu tài nguyên $Request_i$).

1. BẢN CHẤT DEADLOCK VÀ 4 ĐIỀU KIỆN CẦN**A. Định nghĩa Deadlock**

Deadlock (Bế tắc) là tình trạng một tập hợp các tiến trình bị ngưng trệ vô thời hạn vì mỗi tiến trình đang giữ một tài nguyên và chờ đợi cấp phát một tài nguyên khác đang bị giữ bởi một tiến trình khác trong tập hợp đó.

B. 4 Điều kiện Cần để xảy ra Deadlock (Coffman Conditions)

Deadlock CHỈ CÓ THỂ XẢY RA khi đồng thời thỏa mãn cả 4 điều kiện sau:

1. **Loại trừ tương hỗ (Mutual Exclusion):** Ít nhất một tài nguyên được giữ ở chế độ không chia sẻ (chỉ một tiến trình dùng tại một thời điểm).
2. **Giữ và Chờ (Hold and Wait):** Tiến trình đang giữ ít nhất một tài nguyên và đang chờ cấp phát thêm tài nguyên mới mà tài nguyên đó đang bị tiến trình khác giữ.
3. **Không tước đoạt (No Preemption):** Tài nguyên không thể bị tước đoạt từ tiến trình; tài nguyên chỉ được giải phóng tự nguyện bởi tiến trình sau khi hoàn thành tác vụ.
4. **Chờ đợi vòng tròn (Circular Wait):** Tồn tại một tập hợp tiến trình $\{P_0, P_1, \dots, P_n\}$ sao cho P_0 chờ tài nguyên từ P_1 , P_1 chờ P_2 , ..., P_n chờ P_0 .

2. ĐỒ THỊ CẤP PHÁT TÀI NGUYÊN (RAG) VÀ TRẠNG THÁI AN TOÀN**A. Đồ thị Cấp phát Tài nguyên (Resource Allocation Graph - RAG)**

- **Đỉnh (Nodes):** Gồm tập hợp các Tiến trình $P = \{P_1, P_2, \dots\}$ (vẽ bằng hình tròn) và tập hợp các Tài nguyên $R = \{R_1, R_2, \dots\}$ (vẽ bằng hình chữ nhật, bên trong chứa các chấm nhỏ biểu diễn số thực thể / instances).
- **Cung Yêu cầu (Request Edge):** Cung hướng từ tiến trình sang tài nguyên $P_i \rightarrow R_j$ (tiến trình P_i đang chờ tài nguyên R_j).
- **Cung Cấp phát (Assignment Edge):** Cung hướng từ tài nguyên sang tiến trình $R_j \rightarrow P_i$ (tài nguyên R_j đã được cấp cho P_i).

⚠ ĐỪNG NHẦM: Chu trình (Cycle) trong RAG vs Deadlock

- Nếu mỗi tài nguyên chỉ có **1 thực thể (Single Instance)**: Chu trình trong RAG là điều kiện **CẦN VÀ ĐỦ** để xảy ra Deadlock. Tồn tại Cycle $\Leftrightarrow$ Deadlock.
- Nếu tài nguyên có **nhiều thực thể (Multiple Instances)**: Chu trình trong RAG chỉ là điều kiện **CẦN** chứ CHƯA ĐỦ. Tồn tại Cycle $\Rightarrow$ Có thể Deadlock hoặc Không Deadlock.

B. Trạng thái An toàn (Safe State)

Một trạng thái được gọi là **An toàn (Safe State)** nếu HDH có thể tìm được ít nhất một **Chuỗi An toàn (Safe Sequence)** $\langle P_1, P_2, \dots, P_n \rangle$ sao cho với mỗi P_i , tài nguyên P_i yêu cầu có thể được đáp ứng bởi tài nguyên hiện có (Available) cộng với tài nguyên đang được giữ bởi tất cả các P_j trước đó ($j < i$).

3. THUẬT TOÁN BANKER (BANKER'S ALGORITHM) CHUẨN BÀI THI UIT

Thuật toán Banker được dùng để TRÁNH DEADLOCK (Deadlock Avoidance) trong hệ thống có nhiều thực thể tài nguyên.

Các Cấu Trúc Dữ Liệu Cần Khai Báo

- Available[m]**: Vector độ dài m , **Available[j] = k** nghĩa là tài nguyên R_j còn rảnh k thực thể.
- Max[n][m]**: Ma trận $n \times m$, **Max[i][j] = k** nghĩa là P_i yêu cầu tối đa k thực thể R_j .
- Allocation[n][m]**: Ma trận $n \times m$, **Allocation[i][j] = k** nghĩa là P_i đang giữ k thực thể R_j .
- Need[n][m]**: Ma trận $n \times m$, **Need[i][j] = Max[i][j] - Allocation[i][j]** nghĩa là P_i cần thêm k thực thể R_j .



WORKED EXAMPLE: THUẬT TOÁN BANKER VÀ XỬ LÝ REQUEST

Xét hệ thống tại t_0 có 5 tiến trình ($P_0 \rightarrow P_4$) và 4 tài nguyên (A, B, C, D):

PROCESS	ALLOCATION (A B C D)	MAX (A B C D)	AVAILABLE (A B C D)
P0	0 0 1 2	0 0 1 2	1 5 2 0
P1	1 0 0 0	1 7 5 0	-
P2	1 3 5 4	2 3 5 6	-
P3	0 6 3 2	0 6 5 2	-
P4	0 0 1 4	0 6 5 6	-

Yêu cầu:

- Dựng ma trận *Need*. Hệ thống có ở trạng thái an toàn không? Tìm chuỗi an toàn.
- Nếu P_1 yêu cầu thêm $Request_1 = (0, 4, 2, 0)$, hệ thống có thể đáp ứng ngay được không?

a) Dựng ma trận *Need* và Thuật toán An toàn:

Ma trận $Need = Max - Allocation$:

- $Need_0 = (0, 0, 1, 2) - (0, 0, 1, 2) = (0, 0, 0, 0)$

- $Need_1 = (1, 7, 5, 0) - (1, 0, 0, 0) = (0, 7, 5, 0)$
- $Need_2 = (2, 3, 5, 6) - (1, 3, 5, 4) = (1, 0, 0, 2)$
- $Need_3 = (0, 6, 5, 2) - (0, 6, 3, 2) = (0, 0, 2, 0)$
- $Need_4 = (0, 6, 5, 6) - (0, 0, 1, 4) = (0, 6, 4, 2)$

Chạy Thuật toán An toàn: Khởi tạo $Work = Available = (1, 5, 2, 0)$

1. **Duyệt P_0 :** $Need_0(0, 0, 0, 0) \leq Work(1, 5, 2, 0) \Rightarrow P_0$ chạy xong!
 $Work = Work + Allocation_0 = (1, 5, 2, 0) + (0, 0, 1, 2) = (1, 5, 3, 2)$. $Finish[0] = true$.
2. **Duyệt P_2 :** $Need_2(1, 0, 0, 2) \leq Work(1, 5, 3, 2) \Rightarrow P_2$ chạy xong!
 $Work = Work + Allocation_2 = (1, 5, 3, 2) + (1, 3, 5, 4) = (2, 8, 8, 6)$. $Finish[2] = true$.
3. **Duyệt P_3 :** $Need_3(0, 0, 2, 0) \leq Work(2, 8, 8, 6) \Rightarrow P_3$ chạy xong!
 $Work = Work + Allocation_3 = (2, 8, 8, 6) + (0, 6, 3, 2) = (2, 14, 11, 8)$. $Finish[3] = true$.
4. **Duyệt P_4 :** $Need_4(0, 6, 4, 2) \leq Work(2, 14, 11, 8) \Rightarrow P_4$ chạy xong!
 $Work = Work + Allocation_4 = (2, 14, 11, 8) + (0, 0, 1, 4) = (2, 14, 12, 12)$. $Finish[4] = true$.
5. **Duyệt P_1 :** $Need_1(0, 7, 5, 0) \leq Work(2, 14, 12, 12) \Rightarrow P_1$ chạy xong!
 $Work = Work + Allocation_1 = (2, 14, 12, 12) + (1, 0, 0, 0) = (3, 14, 12, 12)$.
 $Finish[1] = true$.

KẾT LUẬN: Tất cả tiến trình đều hoàn tất. Hệ thống ở **Trạng thái AN TOÀN**. Chuỗi an toàn tìm được là: $\langle P_0, P_2, P_3, P_4, P_1 \rangle$.

b) Xử lý Yêu cầu $Request_1 = (0, 4, 2, 0)$ của P_1 :

1. Kiểm tra Bước 1: $Request_1(0, 4, 2, 0) \leq Need_1(0, 7, 5, 0) \rightarrow$ **THỎA MÃN**.
2. Kiểm tra Bước 2: $Request_1(0, 4, 2, 0) \leq Available(1, 5, 2, 0) \rightarrow$ **THỎA MÃN**.
3. Giả lập cấp phát và cập nhật trạng thái mới:
 - $Available_{new} = Available - Request_1 = (1, 5, 2, 0) - (0, 4, 2, 0) = (1, 1, 0, 0)$
 - $Allocation_{1,new} = Allocation_1 + Request_1 = (1, 0, 0, 0) + (0, 4, 2, 0) = (1, 4, 2, 0)$
 - $Need_{1,new} = Need_1 - Request_1 = (0, 7, 5, 0) - (0, 4, 2, 0) = (0, 3, 3, 0)$
4. **Chạy lại Thuật toán An toàn trên trạng thái mới ($Work = 1, 1, 0, 0$):**
 - **Duyệt P_0 :** $Need_0(0, 0, 0, 0) \leq Work(1, 1, 0, 0) \Rightarrow P_0$ xong!
 $Work = Work + Allocation_0 = (1, 1, 0, 0) + (0, 0, 1, 2) = (1, 1, 1, 2)$.
 $Finish[0] = true$.
 - **Duyệt P_2 :** $Need_2(1, 0, 0, 2) \leq Work(1, 1, 1, 2) \Rightarrow P_2$ xong!
 $Work = Work + Allocation_2 = (1, 1, 1, 2) + (1, 3, 5, 4) = (2, 4, 6, 6)$.
 $Finish[2] = true$.
 - **Duyệt P_3 :** $Need_3(0, 0, 2, 0) \leq Work(2, 4, 6, 6) \Rightarrow P_3$ xong!
 $Work = Work + Allocation_3 = (2, 4, 6, 6) + (0, 6, 3, 2) = (2, 10, 9, 8)$.
 $Finish[3] = true$.
 - **Duyệt P_4 :** $Need_4(0, 6, 4, 2) \leq Work(2, 10, 9, 8) \Rightarrow P_4$ xong!
 $Work = Work + Allocation_4 = (2, 10, 9, 8) + (0, 0, 1, 4) = (2, 10, 10, 12)$.
 $Finish[4] = true$.
 - **Duyệt P_1 :** $Need_1(0, 3, 3, 0) \leq Work(2, 10, 10, 12) \Rightarrow P_1$ xong!
 $Work = Work + Allocation_1 = (2, 10, 10, 12) + (1, 4, 2, 0) = (3, 14, 12, 12)$.
 $Finish[1] = true$.

KẾT LUẬN: Tìm được chuỗi an toàn $\langle P_0, P_2, P_3, P_4, P_1 \rangle$. Hệ thống **CÓ THỂ** đáp ứng ngay yêu cầu của P_1 .

4. PHÒNG NGỪA (PREVENTION), PHÁT HIỆN (DETECTION) VÀ KHÔI PHỤC DEADLOCK

A. Phòng Ngừa Deadlock (Deadlock Prevention)

Phòng ngừa Deadlock bằng cách triệt tiêu ít nhất 1 trong 4 điều kiện cần Coffman:

1. **Triệt tiêu Mutual Exclusion:** Biến các tài nguyên thành chia sẻ được (như Spooling đĩa). Tuy nhiên, nhiều tài nguyên (như máy in, ổ đĩa đang ghi) bản chất không thể chia sẻ.
2. **Triệt tiêu Hold and Wait:** Đảm bảo tiến trình chỉ xin tài nguyên khi chưa giữ bất kỳ tài nguyên nào; hoặc phải xin tất cả tài nguyên ngay từ đầu. *Nhược điểm:* Lãng phí tài nguyên, dễ gây đói tài nguyên (Starvation).
3. **Triệt tiêu No Preemption:** Nếu tiến trình đang giữ tài nguyên xin cấp thêm tài nguyên R mà bị từ chối, toàn bộ tài nguyên tiến trình đang giữ sẽ *bị tước đoạt tự động (Preempted)*. Tiến trình bị tạm dừng và chỉ chạy lại khi có đủ cả tài nguyên cũ và mới.
4. **Triệt tiêu Circular Wait (Chiến lược hiệu quả nhất):** Đánh số thứ tự toàn bộ tài nguyên $F : R \rightarrow \mathbb{N}$. Bắt buộc tiến trình phải xin tài nguyên theo thứ tự tăng dần của mã số. Nếu đang giữ R_i , chỉ được xin R_j với $F(R_j) > F(R_i)$.

B. Phát Hiện Deadlock (Deadlock Detection)

Khi hệ thống không dùng Banker hay Prevention, HDH cho phép Deadlock xảy ra nhưng chạy thuật toán kiểm tra định kỳ:

- **Tài nguyên 1 thực thể (Single Instance):** Dùng Đồ thị chờ Wait-For Graph (bỏ các node tài nguyên khỏi RAG). Phát hiện chu trình (Cycle) $\Rightarrow$ Có Deadlock!
- **Tài nguyên nhiều thực thể (Multiple Instances):** Dùng Thuật toán Phát hiện (tương tự Banker nhưng dùng vector `Request[n][m]` trực tiếp thay cho `Need`). Tiến trình nào không thể hoàn thành trong lượt duyệt sẽ thuộc tập bế tắc!

C. Khôi Phục Sau Deadlock (Deadlock Recovery)

Khi đã phát hiện Deadlock, HDH thực hiện 1 trong 2 phương án khôi phục:

1. **Hủy Tiến Trình (Process Termination):**
 - Hủy tất cả các tiến trình bị Deadlock (chỉ phí đắt đỏ).
 - Hủy lần lượt từng tiến trình theo thứ tự ưu tiên cho đến khi hết chu trình bế tắc.
2. **Tước Đoạt Tài Nguyên (Resource Preemption):**
 - **Chọn nạn nhân (Selecting a victim):** Chọn tiến trình có chi phí tước đoạt nhỏ nhất.
 - **Khôi phục (Rollback):** Trả tiến trình nạn nhân về trạng thái an toàn trước đó (Safe state / Checkpoint) và khởi động lại.
 - **Tránh đói tài nguyên (Starvation):** Giới hạn số lần 1 tiến trình bị chọn làm nạn nhân.

Quick Recall Chương 6

- 4 Điều kiện Deadlock: Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait.
- Triệt tiêu Circular Wait: Đánh số thứ tự tài nguyên, xin tăng dần.
- Banker: $Need = Max - Allocation$. Cấp phát $Request$ khi $Request \leq Need$ VÀ $Request \leq Available$.
- Khôi phục Deadlock: Hủy tiến trình hoặc Tước đoạt tài nguyên + Rollback.

CHƯƠNG 07 — TRỌNG TÂM THI CUỐI KỲ

QUẢN LÝ BỘ NHỚ (MEMORY MANAGEMENT)

Địa chỉ Logic vs Vật lý → Placement Algorithms → Kỹ thuật Phân trang (Paging) → EAT & TLB

 Mục Tiêu Chương 7 (Trọng Tâm Cuối Kỳ UIT)

- Phân biệt rạch ròi Địa chỉ Logic (Ảo) và Địa chỉ Vật lý; Nắm các thời điểm ràng buộc địa chỉ (Compile, Load, Execution time).
- Phân biệt Phân mảnh nội (Internal Fragmentation) và Phân mảnh ngoại (External Fragmentation).
- Thạo thuật toán cấp phát bộ nhớ liên tục: First-fit, Best-fit, Worst-fit, Next-fit trên phân vùng cố định & động.
- Hiểu sâu cơ chế Phân trang (Paging), ánh xạ địa chỉ ($p, d \rightarrow f, d$) và cấu trúc Bảng trang nhiều cấp.
- Tính chính xác 100% Thời gian truy xuất hiệu dụng (EAT) khi có bộ đệm TLB.

1. KHÁI NIỆM ĐỊA CHỈ VÀ PHÂN MẢNH BỘ NHỚ

A. Địa chỉ Logic (Virtual Address) vs Địa chỉ Vật lý (Physical Address)

- **Địa chỉ Logic (Ảo):** Địa chỉ được tạo ra bởi CPU khi thực thi chương trình. Tập hợp tất cả các địa chỉ logic tạo thành Không gian địa chỉ logic (Logical Address Space).
- **Địa chỉ Vật lý:** Địa chỉ thực tế trên thanh RAM do bộ nhớ nhận diện. Tập hợp các địa chỉ vật lý tạo thành Không gian địa chỉ vật lý (Physical Address Space).
- **MMU (Memory Management Unit):** Thiết bị phần cứng thực hiện ánh xạ địa chỉ logic thành địa chỉ vật lý tại runtime.

 ĐỪNG NHẦM: Phân Mảnh Nội vs Phân Mảnh Ngoại

LOẠI PHÂN MẢNH	NGUYÊN NHÂN XUẤT HIỆN	ĐỊNH NGHĨA	GIẢI PHÁP KHẮC PHỤC
Phân mảnh nội (Internal Fragmentation)	Bộ nhớ được cấp phát theo các khối kích thước cố định (ví dụ: Frame 4KB). Tiến trình không dùng hết toàn bộ khối.	Phần bộ nhớ thừa bị lãng phí <i>bên trong</i> vùng nhớ đã được cấp phát cho tiến trình.	Giảm kích thước khối cấp phát (Page size nhỏ hơn).
Phân mảnh ngoại (External Fragmentation)	Cấp phát và thu hồi các vùng nhớ động liên tục có kích thước khác nhau.	Tổng bộ nhớ trống đủ cho tiến trình mới nhưng các vùng trống bị <i>chia cắt rời rạc</i> , không liên tục.	Gom cụm bộ nhớ (Compaction) hoặc sử dụng Kỹ thuật Phân trang (Paging).

C. Kỹ Thuật Hoán Đổi (Swapping)

Một tiến trình có thể tạm thời bị **hoán đổi ra (Swap-out)** khỏi bộ nhớ chính RAM để lưu vào vùng nhớ thứ cấp (Backing Store / Swap Space trên đĩa) khi hệ thống thiếu RAM, và sau đó được **hoán đổi vào (Swap-in)** lại RAM khi cần tiếp tục thực thi:

- **Tác dụng:** Cho phép tổng không gian nhớ vật lý của các tiến trình vượt quá dung lượng thực tế của thanh RAM, tăng mức độ đa chương (Degree of Multiprogramming).
- **Chi phí:** Thời gian truyền dữ liệu qua đĩa (Disk Transfer Time) chiếm tỷ trọng lớn trong thời gian chuyển ngữ cảnh. Trong các hệ điều hành hiện đại, Swapping thường kết hợp chặt chẽ với Kỹ thuật Phân trang theo yêu cầu (Paging-based Swapping).

2. CÁC THUẬT TOÁN PLACEMENT (FIRST-FIT, BEST-FIT, WORST-FIT, NEXT-FIT)

Khi các phân vùng nhớ rảnh có kích thước khác nhau, thuật toán Placement quyết định chọn phân vùng nào cho tiến trình xin cấp phát S KB:

- **First-fit (Khớp đầu tiên):** Cấp phát phân vùng trống *đầu tiên* đủ lớn (duyệt từ đầu danh sách).
- **Best-fit (Khớp tốt nhất):** Cấp phát phân vùng trống *nhỏ nhất* nhưng đủ lớn (tạo ra vùng thừa nhỏ nhất, đòi hỏi tìm toàn bộ danh sách).
- **Worst-fit (Khớp xấu nhất):** Cấp phát phân vùng trống *lớn nhất* (tạo ra vùng thừa lớn nhất còn lại).
- **Next-fit (Khớp tiếp theo):** Giống First-fit nhưng bắt đầu tìm từ *vị trí phân vùng vừa cấp phát cho tiến trình trước đó*.



WORKED EXAMPLE: THUẬT TOÁN PLACEMENT (CHUẨN ĐỀ THI UIT)

Cho các phân vùng bộ nhớ theo thứ tự: 100KB, 500KB, 200KB, 300KB, 600KB .

Xác định phân vùng được cấp phát cho lần lượt các tiến trình có kích thước: P1 (212KB), P2 (417KB), P3 (112KB), P4 (426KB) theo thuật toán First-fit, Best-fit, Worst-fit.

Lời giải chi tiết:

- **First-fit:**
 - P1 (212KB): Cấp vào phân vùng 500KB (còn thừa 288KB).
 - P2 (417KB): Cấp vào phân vùng 600KB (còn thừa 183KB).
 - P3 (112KB): Cấp vào phân vùng 200KB (còn thừa 88KB).
 - P4 (426KB): Duyệt qua 100KB, 288KB, 88KB, 300KB, 183KB → **Không có phân vùng nào đủ → P4 phải chờ!**
- **Best-fit:**
 - P1 (212KB): Cấp vào phân vùng 300KB (thừa 88KB - nhỏ nhất đủ lớn).
 - P2 (417KB): Cấp vào phân vùng 500KB (thừa 83KB).
 - P3 (112KB): Cấp vào phân vùng 200KB (thừa 88KB).
 - P4 (426KB): Cấp vào phân vùng 600KB (thừa 174KB) → **Tất cả tiến trình đều được cấp phát!**
- **Worst-fit:**
 - P1 (212KB): Cấp vào phân vùng lớn nhất 600KB (thừa 388KB).
 - P2 (417KB): Cấp vào phân vùng lớn nhất 500KB (thừa 83KB).
 - P3 (112KB): Cấp vào phân vùng lớn nhất 388KB (thừa 276KB).
 - P4 (426KB): Không có phân vùng nào đủ → **P4 phải chờ!**

3. KỸ THUẬT PHÂN TRANG (PAGING) VÀ ÁNH XẠ ĐỊA CHỈ

A. Cơ chế Phân trang

Phân trang cho phép không gian địa chỉ vật lý của tiến trình không cần nằm liên tục nhau trong RAM, giải quyết hoàn toàn Phân mảnh ngoại.

- Bộ nhớ vật lý RAM được chia thành các khối cố định gọi là **Khung trang (Frame)**.
- Bộ nhớ logic được chia thành các khối có cùng kích thước gọi là **Trang (Page)**.

B. Cấu trúc Địa chỉ Logic và Chuyển đổi Địa chỉ

Địa chỉ logic gồm 2 phần: chỉ số trang (p) và độ dời (d).

Giả sử địa chỉ logic có m bit, kích thước trang là 2^n bytes $\Rightarrow n$ bit cuối dành cho độ dời d , $m - n$ bit đầu dành cho chỉ số trang p .

⚠ QUY TẮC VÀNG CHUYỂN ĐỔI ĐỊA CHỈ PHÂN TRANG

Địa chỉ Logic (p, d) $\xrightarrow{\text{Bảng trang}}$ Địa chỉ Vật lý (f, d)

ĐỘ DỜI d KHÔNG BAO GIỜ THAY ĐỔI khi chuyển từ địa chỉ ảo sang địa chỉ vật lý! Chỉ có p (chỉ số trang) được tra cứu trong Bảng trang (Page Table) để thay bằng f (chỉ số khung trang).

C. Thời gian Truy xuất Hiệu dụng EAT với TLB

Bộ đệm TLB (Translation Lookaside Buffer) là bộ nhớ cache phần cứng siêu nhanh lưu trữ các cặp (p, f) hay dùng.

📐 CÔNG THỨC EAT VÀ CÁCH THỂ SỐ CHUẨN ĐỀ THI UIT

Gọi α là Tỷ lệ tìm thấy trong TLB (Hit ratio), t_{RAM} là thời gian truy xuất RAM, t_{TLB} là thời gian tìm trong TLB.

Công thức tổng quát:

$$EAT = \alpha \cdot (t_{TLB} + t_{RAM}) + (1 - \alpha) \cdot (t_{TLB} + 2 \cdot t_{RAM})$$

$$EAT = t_{TLB} + (2 - \alpha) \cdot t_{RAM}$$

Trường hợp đặc biệt: Nếu đề bài ghi "Thời gian tìm trong TLB xem như bằng 0" ($t_{TLB} = 0$), công thức rút gọn là:

$$EAT = (2 - \alpha) \cdot t_{RAM}$$

4. BÀI TẬP WORKED EXAMPLES CHUẨN ĐỀ THI UIT

📄 WORKED EXAMPLE 1: ÁNH XẠ ĐỊA CHỈ VÀ TÍNH BIT

Bài toán: Xét một không gian địa chỉ ảo gồm 32 trang, mỗi trang có kích thước 2KB (2048 bytes), ánh xạ vào bộ nhớ vật lý có 64 khung trang.

a) Địa chỉ logic gồm bao nhiêu bit? Địa chỉ vật lý gồm bao nhiêu bit?

b) Cho địa chỉ logic dạng số nguyên là $D = 4600$. Hãy xác định chỉ số trang p , độ dời d và địa chỉ vật lý tương ứng nếu trang p được nạp vào khung trang $f = 9$.

Lời giải chi tiết:

1. Tính số bit Địa chỉ Logic:

- Kích thước trang $2KB = 2^{11}$ bytes $\Rightarrow$ Số bit cho Offset $d = 11$ bits.
- Số trang $32 = 2^5$ trang $\Rightarrow$ Số bit cho chỉ số trang $p = 5$ bits.
- Vậy Tổng số bit địa chỉ logic = $5 + 11 = 16$ bits.

2. Tính số bit Địa chỉ Vật lý:

- Số khung trang $64 = 2^6$ khung $\Rightarrow$ Số bit cho khung trang $f = 6$ bits.
- Offset d không đổi = 11 bits.
- Vậy Tổng số bit địa chỉ vật lý = $6 + 11 = 17$ bits.

3. Tách địa chỉ logic $D = 4600$:

- Chỉ số trang $p = \lfloor 4600/2048 \rfloor = 2$.
- Độ dời $d = 4600 \pmod{2048} = 4600 - 2 \times 2048 = 504$.

4. Tính Địa chỉ Vật lý:

Trang $p = 2$ được nạp vào Khung trang $f = 9$.

Địa chỉ vật lý = $f \times \text{Page_Size} + d = 9 \times 2048 + 504 = 18432 + 504 = 18936$.



WORKED EXAMPLE 2: TÍNH THỜI GIAN TRUY XUẤT HIỆU DỤNG EAT

Bài toán EAT:

- Cho $t_{RAM} = 100$ ns, $t_{TLB} = 20$ ns, $\alpha = 80\%$. Tính EAT.
- Cho $t_{RAM} = 200$ ns, thời gian tra TLB xem như bằng 0, $\alpha = 75\%$. Tính EAT.
- Cho $t_{RAM} = 100$ ns, $t_{TLB} = 20$ ns. Hệ thống yêu cầu $EAT \leq 130$ ns. Hỏi tỷ lệ hit ratio α tối thiểu của TLB phải đạt bao nhiêu %?

Lời giải chi tiết:

1. a) Thế số công thức đầy đủ:

$$EAT = t_{TLB} + (2 - \alpha)t_{RAM} = 20 + (2 - 0.8) \times 100 = 20 + 1.2 \times 100 = 140 \text{ ns.}$$

2. b) Thế số công thức rút gọn ($t_{TLB} = 0$):

$$EAT = (2 - \alpha)t_{RAM} = (2 - 0.75) \times 200 = 1.25 \times 200 = 250 \text{ ns.}$$

3. c) Bài toán ngược tìm α :

$$EAT = 20 + (2 - \alpha) \times 100 \leq 130$$

$$\Leftrightarrow 100(2 - \alpha) \leq 110 \Leftrightarrow 2 - \alpha \leq 1.1 \Leftrightarrow \alpha \geq 0.9 = 90\%.$$

Vậy Tỷ lệ hit ratio của TLB tối thiểu phải đạt **90%**.



Quick Recall Chương 7

- Internal Fragmentation (trong khối) vs External Fragmentation (ngoài khối rãnh).
- Placement: Best-fit tạo mảnh thừa nhỏ nhất; Worst-fit chọn khối lớn nhất.
- Paging: $Logical(p, d) \rightarrow Physical(f, d)$. Độ dời d KHÔNG THAY ĐỔI.
- $EAT = t_{TLB} + (2 - \alpha)t_{RAM}$.

CHƯƠNG 08 — TRỌNG TÂM THI CUỐI KỲ

BỘ NHỚ ẢO (VIRTUAL MEMORY)

Demand Paging → Page Fault → Thuật toán thay thế trang (FIFO, OPT, LRU) → Trì trệ Thrashing

 Mục Tiêu Chương 8 (Trọng Tâm Cuối Kỳ UIT)

- Hiểu bản chất Bộ nhớ ảo và cơ chế Nạp trang theo yêu cầu (Demand Paging).
- Thuộc lòng quy trình 6 bước xử lý ngắt Lỗi trang (Page Fault).
- Vẽ chính xác 100% bảng theo dõi thay thế trang cho các thuật toán: FIFO, OPT, LRU.
- Hiểu Nghịch lý Belady (Belady's Anomaly) và tại sao LRU/OPT không bị ảnh hưởng.
- Hiểu hiện tượng Trì trệ hệ thống (Thrashing) và mô hình Tập làm việc (Working-Set Model).

1. KHÁI NIỆM BỘ NHỚ ẢO VÀ LỖI TRANG (PAGE FAULT)

A. Bản chất Bộ nhớ ảo và Demand Paging

Bộ nhớ ảo cho phép thực thi một chương trình mà **không cần nạp toàn bộ chương trình đó vào RAM vật lý**. Kỹ thuật phổ biến nhất là Nạp trang theo yêu cầu (Demand Paging): Trang nào được CPU truy xuất tới mới được nạp từ đĩa cứng vào RAM.

 6 BƯỚC XỬ LÝ LỖI TRANG (PAGE FAULT WALKTHROUGH)

1. CPU truy xuất một địa chỉ logic. Bảng trang kiểm tra bit Valid/Invalid. Nếu bit = Invalid → Báo hiệu **Page Fault**.
2. Phần cứng phát ngắt Trap chuyển quyền cho HDH.
3. HDH kiểm tra xem yêu cầu truy xuất có hợp lệ không. Nếu không hợp lệ → Hủy tiến trình (Segment fault). Nếu hợp lệ → Tìm một khung trang trống (Free frame) trong RAM.
4. HDH định vị trang tương ứng trên đĩa cứng và thực hiện đọc trang vào khung trang trống vừa tìm.
5. Khi đọc đĩa hoàn tất, HDH cập nhật Bảng trang (đổi bit Invalid → Valid) và ghi lại số khung trang f .
6. HDH khôi phục trạng thái CPU và tái thực thi lại chính câu lệnh vừa gây ra Page Fault.

2. CÁC THUẬT TOÁN THAY THẾ TRANG (PAGE REPLACEMENT)

Khi xảy ra Lỗi trang nhưng RAM đã ĐẦY (không còn khung trang trống), HDH phải chọn một trang trong RAM làm **Nạn nhân (Victim page)** để hoán vị ra đĩa (Swap out) nhường chỗ cho trang mới (Swap in).

⚠ ĐỪNG NHẦM: FIFO vs OPT vs LRU

THUẬT TOÁN	NGUYÊN LÝ CHỌN VICTIM	ĐẶC ĐIỂM & HIỆU NĂNG	NGHỊCH LÝ BELADY?
FIFO (First-In, First-Out)	Thay thế trang được nạp vào RAM <i>sớm nhất</i> .	Dễ cài đặt bằng Queue, nhưng hiệu năng kém. Cần lưu thời điểm nạp.	CÓ BỊ (Tăng số Frame có thể làm tăng số Page Fault!).
OPT (Optimal - Tối ưu)	Thay thế trang sẽ <i>không được sử dụng trong thời gian lâu nhất ở TƯƠNG LAI</i> .	Cho số Page Fault nhỏ nhất tuyệt đối. Không cài đặt thực tế được (chỉ dùng làm chuẩn so sánh).	KHÔNG BỊ (Thuộc lớp Stack Algorithms).
LRU (Least Recently Used)	Thay thế trang <i>chưa được sử dụng trong thời gian lâu nhất ở QUÁ KHỨ</i> .	Hiệu năng rất tốt, xấp xỉ OPT. Được ứng dụng thực tế phổ biến. Đòi hỏi phần cứng hỗ trợ (Counter/Stack).	KHÔNG BỊ (Thuộc lớp Stack Algorithms).

3. BÀI TẬP WORKED EXAMPLE THAY THẾ TRANG CHUẨN THI UIT

📄 WORKED EXAMPLE: THAY THẾ TRANG FIFO, OPT, LRU

Cho chuỗi truy xuất trang (Reference String):

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

Giả sử hệ thống có 4 Khung trang (Frames), ban đầu tất cả các khung trang đều trống.

Tính tổng số Lỗi trang (Page Faults) xảy ra khi sử dụng các thuật toán: a) LRU, b) FIFO, c) OPT.

a) Giải Thuật LRU (Least Recently Used - 4 Khung trang):

Bảng diễn biến nạp từng bước và chọn Victim:

MỐC REF	TRANG GỌI	KHUN G 1	KHUN G 2	KHUN G 3	KHUN G 4	TRẠNG THÁI (HIT / FAULT)	GIẢI THÍCH VICTIM
1	1	1	-	-	-	Fault (*) [PF=1]	Khung trống 1
2	2	1	2	-	-	Fault (*) [PF=2]	Khung trống 2
3	3	1	2	3	-	Fault (*) [PF=3]	Khung trống 3
4	4	1	2	3	4	Fault (*) [PF=4]	Khung trống 4
5	2	1	2	3	4	Hit	Trang 2 đã có sẵn
6	1	1	2	3	4	Hit	Trang 1 đã có sẵn
7	5	1	2	5	4	Fault (*) [PF=5]	Trang 3 ít dùng nhất trong quá khứ → Thay 3 bằng 5
8	6	1	2	5	6	Fault (*) [PF=6]	Trang 4 ít dùng nhất trong quá khứ → Thay 4 bằng 6
9	2	1	2	5	6	Hit	Trang 2 đã có sẵn
10	1	1	2	5	6	Hit	Trang 1 đã có sẵn

MỐC REF	TRANG GỌI	KHUN G 1	KHUN G 2	KHUN G 3	KHUN G 4	TRẠNG THÁI (HIT / FAULT)	GIẢI THÍCH VICTIM
11	2	1	2	5	6	Hit	Trang 2 đã có sẵn
12	3	1	2	3	6	Fault (*) [PF=7]	Trang 5 ít dùng nhất trong quá khứ → Thay 5 bằng 3
13	7	1	2	3	7	Fault (*) [PF=8]	Trang 6 ít dùng nhất trong quá khứ → Thay 6 bằng 7
14	6	6	2	3	7	Fault (*) [PF=9]	Trang 1 ít dùng nhất trong quá khứ → Thay 1 bằng 6
15	3	6	2	3	7	Hit	Trang 3 đã có sẵn
16	2	6	2	3	7	Hit	Trang 2 đã có sẵn
17	1	6	2	3	1	Fault (*) [PF=10]	Trang 7 ít dùng nhất trong quá khứ → Thay 7 bằng 1
18	2	6	2	3	1	Hit	Trang 2 đã có sẵn
19	3	6	2	3	1	Hit	Trang 3 đã có sẵn
20	6	6	2	3	1	Hit	Trang 6 đã có sẵn

Tổng số Lỗi trang của LRU = 10 lỗi.

b) Giải Thuật FIFO (First-In, First-Out - 4 Khung trang):

Thay thế theo thứ tự nạp vào sớm nhất (Queue F1 → F2 → F3 → F4 → F1...):

- Ref 1,2,3,4: Nạp vào [1,2,3,4] → 4 Page Faults (PF=4).
- Ref 2, 1: Hit.
- Ref 5: Thay F1 (trang 1) → [5,2,3,4] (PF=5).
- Ref 6: Thay F2 (trang 2) → [5,6,3,4] (PF=6).
- Ref 2: Thay F3 (trang 3) → [5,6,2,4] (PF=7).
- Ref 1: Thay F4 (trang 4) → [5,6,2,1] (PF=8).
- Ref 2: Hit (có sẵn).
- Ref 3: Thay F1 (trang 5) → [3,6,2,1] (PF=9).
- Ref 7: Thay F2 (trang 6) → [3,7,2,1] (PF=10).
- Ref 6: Thay F3 (trang 2) → [3,7,6,1] (PF=11).
- Ref 3: Hit (có sẵn).
- Ref 2: Thay F4 (trang 1) → [3,7,6,2] (PF=12).
- Ref 1: Thay F1 (trang 3) → [1,7,6,2] (PF=13).
- Ref 2: Hit.
- Ref 3: Thay F2 (trang 7) → [1,3,6,2] (PF=14).
- Ref 6: Hit.

Tổng số Lỗi trang của FIFO = 14 lỗi.

c) Giải Thuật OPT (Optimal Page Replacement - 4 Khung trang):

Thay thế trang không được dùng trong thời gian lâu nhất ở tương lai:

- Ref 1,2,3,4: Nạp vào [1,2,3,4] → 4 Page Faults (PF=4).
- Ref 2, 1: Hit.
- Ref 5: Trang 4 không còn dùng nữa trong tương lai → Thay 4 bằng 5 → [1,2,3,5] (PF=5).
- Ref 6: Trang 5 không dùng nữa → Thay 5 bằng 6 → [1,2,3,6] (PF=6).
- Ref 2, 1, 2, 3: Hit (tất cả 1,2,3 đều có sẵn).
- Ref 7: Nhìn vào tương lai **6, 3, 2, 1, 2, 3, 6** : trang 1 dùng xa nhất (tại mốc 17) → Thay 1 bằng 7 → [7,2,3,6] (PF=7).
- Ref 6, 3, 2: Hit.
- Ref 1: Trang 7 không dùng nữa → Thay 7 bằng 1 → [1,2,3,6] (PF=8).
- Ref 2, 3, 6: Hit.

Tổng số Lỗi trang của OPT = 8 lỗi.

4. TRÌ TRỆ HỆ THỐNG (THRASHING) VÀ WORKING-SET MODEL

Khi số lượng tiến trình tăng quá cao, số khung trang rảnh cho mỗi tiến trình quá ít → Tiến trình liên tục bị Lỗi trang. HDH dành 99% thời gian để Swap trang ra/vào đĩa cứng thay vì thực thi tính toán. Hiện tượng này gọi là **Thrashing (Trì trệ hệ thống)**.

Giải pháp: Mô hình Tập làm việc (Working-Set Model) dựa trên tính địa phương (Locality of Reference). Cấp phát đủ số khung trang nằm trong cửa sổ Working Set Δ để ngăn ngừa Thrashing.

Quick Recall Chương 8

- Page Fault xảy ra khi bit Valid/Invalid = Invalid.
- FIFO bị Nghịch lý Belady (tăng Frame tăng Page fault). LRU/OPT không bị.
- LRU nhìn về QUÁ KHỨ, OPT nhìn vào TƯƠNG LAI.
- Thrashing: CPU lãng phí thời gian chỉ để hoán vị trang liên tục.

PHẦN REVIEW ĐẶC BIỆT — CUỐI KỲ

FINAL MASTER REVIEW & 02 ĐỀ THI MÔ PHỎNG CUỐI KỲ CHUẨN UIT

Hệ thống hóa Ch5-Ch8 → Bẫy trắc nghiệm → 02 Đề thi thử Cuối kỳ hoàn chỉnh có đáp án

1. BẢN ĐỒ KIẾN THỨC BẮT BUỘC THI CUỐI KỲ (CH5 - CH8)

SYSTEM CHECKLIST CUỐI KỲ IT007

- **Chương 5 (Đồng bộ):** 3 Điều kiện CS (Mutual Exclusion, Progress, Bounded Waiting), Peterson, Semaphore `wait()` (P) và `signal()` (V). Viết code Semaphore cho sơ đồ phụ thuộc DAG và bài toán biến $X \leq 20$.
- **Chương 6 (Deadlock):** 4 Điều kiện cần Coffman, Đồ thị RAG (Single instance: Cycle $\Leftrightarrow$ Deadlock vs Multiple instances: Cycle $\Rightarrow$ Chưa chắc Deadlock), Thuật toán Banker ($Need = Max - Allocation$, Kiểm tra Safe State, Xử lý $Request \leq Need \wedge Request \leq Available$).
- **Chương 7 (Bộ nhớ):** Phân mảnh nội vs Phân mảnh ngoại, Placement (First-fit, Best-fit, Worst-fit, Next-fit), Phân trang $Logical(p, d) \rightarrow Physical(f, d)$ (d không đổi!), $EAT = t_{TLB} + (2 - \alpha)t_{RAM}$.
- **Chương 8 (Bộ nhớ ảo):** Demand Paging, 6 Bước xử lý Page Fault, Thay thế trang (FIFO, OPT, LRU), Nghịch lý Belady trên FIFO, Hiện tượng Trì trệ Thrashing.

2. ĐỀ THI MÔ PHỎNG CUỐI KỲ SỐ 01 (THỜI GIAN: 90 PHÚT)

PHẦN 1: TRẮC NGHIỆM LÝ THUYẾT (3.0 ĐIỂM)

Câu 1: Điều kiện nào sau đây KHÔNG PHẢI là một trong 4 điều kiện cần để xảy ra Deadlock?

- A. Loại trừ tương hỗ (Mutual Exclusion).
- B. Giữ và Chờ (Hold and Wait).
- C. Tước đoạt tài nguyên (Preemption).
- D. Chờ đợi vòng tròn (Circular Wait).

Câu 2: Trong cơ chế phân trang (Paging), địa chỉ logic 16-bit với kích thước trang 4KB (4096 bytes) sẽ có số bit dành cho chỉ số trang p và độ dời d lần lượt là:

- A. $p = 4$ bits, $d = 12$ bits.
- B. $p = 8$ bits, $d = 8$ bits.
- C. $p = 12$ bits, $d = 4$ bits.
- D. $p = 6$ bits, $d = 10$ bits.

Câu 3: Thuật toán thay thế trang nào sau đây có khả năng mắc phải Nghịch lý Belady (Belady's Anomaly)?

- A. LRU (Least Recently Used).
- B. OPT (Optimal).
- C. FIFO (First-In, First-Out).
- D. Cả LRU và FIFO.

PHẦN 2: TỰ LUẬN ĐỒNG BỘ TIẾN TRÌNH (2.5 ĐIỂM)

Câu 4: Cho biến nguyên X được chia sẻ bởi 2 tiến trình P_1 và P_2 cùng thực hiện đoạn code sau:


```
do {
    X = X + 1;
    if (X == 20) X = 0;
} while (TRUE);
```

Giả sử X ban đầu có giá trị 0.

- Chứng minh rằng giá trị của X có thể vượt quá 20 nếu không có đồng bộ.
- Hãy sử dụng Semaphore để sửa chữa đoạn chương trình trên, đảm bảo giá trị của X không bao giờ vượt quá 20.

PHẦN 3: TỰ LUẬN DEADLOCK & BANKER (2.5 ĐIỂM)

Câu 5: Xét một hệ thống tại thời điểm t_0 có 5 tiến trình ($P_0 \rightarrow P_4$) và 3 loại tài nguyên (A, B, C với số lượng thực thể tương ứng là 10, 5, 7):

PROCESS	ALLOCATION (A B C)	MAX (A B C)	AVAILABLE (A B C)
P0	0 1 0	7 5 3	3 3 2
P1	2 0 0	3 2 2	-
P2	3 0 2	9 0 2	-
P3	2 1 1	2 2 2	-
P4	0 0 2	4 3 3	-

Yêu cầu:

- Tìm ma trận *Need*. Kiểm tra xem hệ thống có ở trạng thái an toàn không? Viết ra chuỗi an toàn.
- Nếu tiến trình P_1 đưa ra yêu cầu $Request_1 = (1, 0, 2)$, hệ thống có chấp nhận cấp phát ngay không? Tại sao?

PHẦN 4: TỰ LUẬN THAY THẾ TRANG & BỘ NHỚ (2.0 ĐIỂM)

Câu 6: Xét chuỗi truy xuất trang: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1. Giả sử hệ thống có 3 Khung trang (Frames) ban đầu trống. Tính số Lỗi trang (Page Fault) xảy ra khi dùng thuật toán LRU và OPT.

3. ĐÁP ÁN CHI TIẾT ĐỀ THI MÔ PHÒNG CUỐI KỲ SỐ 01

PHẦN 1: TRẮC NGHIỆM

- Câu 1: C** — Điều kiện đúng là "No Preemption" (Không tước đoạt). "Tước đoạt" là biện pháp triệt tiêu Deadlock.
- Câu 2: A** — Page size $4KB = 2^{12} \Rightarrow d = 12$ bits. Tổng 16 bits $\Rightarrow p = 16 - 12 = 4$ bits.
- Câu 3: C** — Chỉ có FIFO mắc Nghịch lý Belady. LRU và OPT là Stack Algorithms nên không bao giờ bị.

PHẦN 2: LỜI GIẢI CÂU 4 (ĐỒNG BỘ SEMAPHORE)

1. a) Chứng minh X có thể vượt quá 20:

Giả sử $X = 19$. P_1 chạy $X = X + 1$ (đã tính xong $X = 20$ trong thanh ghi nhưng chưa kiểm tra `if`). P_2 nhảy vào chạy $X = X + 1$ làm X thành 21. Sau đó P_1 mới kiểm tra `if (X == 20)` → sai vì X đã là 21 → X không được gán lại về 0 và tiếp tục tăng vượt quá 20.

2. b) Sửa bằng Semaphore:

Khai báo Mutex Semaphore `mutex = 1` để đảm bảo độc quyền truy xuất đoạn code kiểm tra và tăng X :

```
semaphore mutex = 1;

// Trong mỗi tiến trình:
do {
    wait(mutex);
    X = X + 1;
    if (X == 20) X = 0;
    signal(mutex);
} while (TRUE);
```

PHẦN 3: LỜI GIẢI CÂU 5 (BANKER'S ALGORITHM)

1. a) Ma trận $Need = Max - Allocation$:

$Need_0 = (7, 4, 3)$, $Need_1 = (1, 2, 2)$, $Need_2 = (6, 0, 0)$, $Need_3 = (0, 1, 1)$, $Need_4 = (4, 3, 1)$.

2. Chạy Thuật toán An toàn ($Work = 3, 3, 2$):

- $Need_1(1, 2, 2) \leq Work(3, 3, 2) \Rightarrow P_1$ xong!

$Work = Work + Alloc_1 = (3, 3, 2) + (2, 0, 0) = (5, 3, 2)$.

- $Need_3(0, 1, 1) \leq Work(5, 3, 2) \Rightarrow P_3$ xong!

$Work = Work + Alloc_3 = (5, 3, 2) + (2, 1, 1) = (7, 4, 3)$.

- $Need_4(4, 3, 1) \leq Work(7, 4, 3) \Rightarrow P_4$ xong!

$Work = Work + Alloc_4 = (7, 4, 3) + (0, 0, 2) = (7, 4, 5)$.

- $Need_0(7, 4, 3) \leq Work(7, 4, 5) \Rightarrow P_0$ xong!

$Work = Work + Alloc_0 = (7, 4, 5) + (0, 1, 0) = (7, 5, 5)$.

- $Need_2(6, 0, 0) \leq Work(7, 5, 5) \Rightarrow P_2$ xong!

$Work = Work + Alloc_2 = (7, 5, 5) + (3, 0, 2) = (10, 5, 7)$.

Chuỗi an toàn: $\langle P_1, P_3, P_4, P_0, P_2 \rangle \Rightarrow$ Hệ thống AN TOÀN.

3. b) Xử lý $Request_1 = (1, 0, 2)$:

1. $Request_1(1, 0, 2) \leq Need_1(1, 2, 2)$ (Thỏa mãn).

2. $Request_1(1, 0, 2) \leq Available(3, 3, 2)$ (Thỏa mãn).

3. Giả lập cấp phát: $Available_{new} = (2, 3, 0)$, $Alloc_{1,new} = (3, 0, 2)$, $Need_{1,new} = (0, 2, 0)$.

4. Chạy thuật toán an toàn trên trạng thái mới → Tìm được chuỗi an toàn $\langle P_1, P_3, P_4, P_0, P_2 \rangle$.

KẾT LUẬN: CÓ THỂ đáp ứng ngay yêu cầu của P_1 .

PHẦN 4: LỜI GIẢI CÂU 6 (THAY THẾ TRANG LRU & OPT - 3 KHUNG TRANG)

Chuỗi truy xuất (20 refs): 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (3 Khung trang ban đầu trống)

1. Thuật toán LRU (Least Recently Used):

- Ref 7, 0, 1: Nạp vào [7, 0, 1] → 3 Faults (PF=3).
- Ref 2: Trang 7 ít dùng nhất → Thay 7 bằng 2 → [2, 0, 1] (PF=4).
- Ref 0: Hit.
- Ref 3: Trang 1 ít dùng nhất → Thay 1 bằng 3 → [2, 0, 3] (PF=5).
- Ref 0: Hit.
- Ref 4: Trang 2 ít dùng nhất → Thay 2 bằng 4 → [4, 0, 3] (PF=6).
- Ref 2: Trang 3 ít dùng nhất → Thay 3 bằng 2 → [4, 0, 2] (PF=7).
- Ref 3: Trang 0 ít dùng nhất → Thay 0 bằng 3 → [4, 3, 2] (PF=8).
- Ref 0: Trang 4 ít dùng nhất → Thay 4 bằng 0 → [0, 3, 2] (PF=9).
- Ref 3, 2: Hit.
- Ref 1: Trang 0 ít dùng nhất → Thay 0 bằng 1 → [1, 3, 2] (PF=10).
- Ref 2: Hit.
- Ref 0: Trang 3 ít dùng nhất → Thay 3 bằng 0 → [1, 0, 2] (PF=11).
- Ref 1: Hit.
- Ref 7: Trang 2 ít dùng nhất → Thay 2 bằng 7 → [1, 0, 7] (PF=12).
- Ref 0, 1: Hit.

Tổng số Lỗi trang của LRU (3 frames) = 12 lỗi.

2. Thuật toán OPT (Optimal Page Replacement):

- Ref 7, 0, 1: Nạp vào [7, 0, 1] → 3 Faults (PF=3).
- Ref 2: Trang 7 dùng xa nhất ở tương lai (mốc 18) → Thay 7 bằng 2 → [2, 0, 1] (PF=4).
- Ref 0: Hit.
- Ref 3: Trang 1 dùng xa nhất ở tương lai (mốc 14) → Thay 1 bằng 3 → [2, 0, 3] (PF=5).
- Ref 0: Hit.
- Ref 4: Trang 0 dùng xa nhất ở tương lai (mốc 11) → Thay 0 bằng 4 → [2, 4, 3] (PF=6).
- Ref 2, 3: Hit.
- Ref 0: Trang 4 không dùng nữa → Thay 4 bằng 0 → [2, 0, 3] (PF=7).
- Ref 3, 2: Hit.
- Ref 1: Trang 3 không dùng nữa → Thay 3 bằng 1 → [2, 0, 1] (PF=8).
- Ref 2, 0, 1: Hit.
- Ref 7: Trang 2 không dùng nữa → Thay 2 bằng 7 → [7, 0, 1] (PF=9).
- Ref 0, 1: Hit.

Tổng số Lỗi trang của OPT (3 frames) = 9 lỗi.

4. ĐỀ THI MÔ PHÒNG CUỐI KỲ SỐ 02 (THỜI GIAN: 90 PHÚT)

PHẦN 1: TRẮC NGHIỆM LÝ THUYẾT (3.0 ĐIỂM)

Câu 1: Cho $t_{RAM} = 100$ ns, $t_{TLB} = 20$ ns và tỷ lệ tìm thấy TLB $\alpha = 80\%$. Thời gian truy xuất hiệu dụng EAT bằng:

- A. 120 ns.
- B. 140 ns.
- C. 200 ns.
- D. 180 ns.

Câu 2: Giải pháp Vùng tranh chấp (Critical Section) đòi hỏi điều kiện Tiến triển (Progress). Khái niệm Tiến triển có nghĩa là:

- A. Không tiến trình nào bị kẹt vô hạn.
- B. Chỉ những tiến trình muốn vào CS và không nằm trong Remainder section mới được tham gia chọn tiến trình tiếp theo vào CS.
- C. Chỉ 1 tiến trình được ở trong CS tại 1 thời điểm.
- D. Tốc độ CPU không làm ảnh hưởng thuật toán.

Câu 3: Trạng thái An toàn (Safe State) trong Thuật toán Banker đảm bảo:

- A. Chắc chắn không bao giờ xảy ra Deadlock.
- B. Hệ thống đang bị Deadlock.
- C. Không có tiến trình nào xin thêm tài nguyên.
- D. Tất cả tài nguyên đều đã cấp hết.

PHẦN 2: TỰ LUẬN ĐỒNG BỘ PRODUCER-CONSUMER (3.5 ĐIỂM)

Câu 4: Xét bài toán Nhà sản xuất - Người tiêu thụ (Producer - Consumer) chia sẻ một bộ đệm vòng tròn gồm $N = 5$ phần tử.

- a) Trình bày các nguy cơ Race Condition nếu không đồng bộ.
- b) Khai báo các Semaphore và viết mã C đồng bộ hoàn chỉnh cho Producer và Consumer bằng POSIX Semaphores.

PHẦN 3: TỰ LUẬN BỘ NHỚ VÀ ÁNH XẠ ĐỊA CHỈ (3.5 ĐIỂM)

Câu 5: Một hệ thống phân trang có kích thước trang $4KB = 2^{12}$ bytes, địa chỉ ảo 32-bit.

- a) Tính số bit cho chỉ số trang p và độ dời d . Bảng trang có tối đa bao nhiêu mục (entries)?
- b) Cho địa chỉ ảo dạng thập phân $V = 16390$. Xác định chỉ số trang p và offset d . Nếu trang p được ánh xạ vào khung trang (frame) $f = 5$, hãy tính địa chỉ vật lý tương ứng.

5. ĐÁP ÁN CHI TIẾT ĐỀ THI MÔ PHỎNG CUỐI KỲ SỐ 02

PHẦN 1: ĐÁP ÁN TRẮC NGHIỆM

- **Câu 1: B** — $EAT = t_{TLB} + (2 - \alpha)t_{RAM} = 20 + (2 - 0.8) \times 100 = 20 + 120 = 140$ ns.
- **Câu 2: B** — Đúng định nghĩa điều kiện Progress theo Silberschatz/UIT.
- **Câu 3: A** — Safe State đảm bảo tồn tại ít nhất 1 Safe Sequence $\Rightarrow$ chắc chắn KHÔNG bị Deadlock.

PHẦN 2: LỜI GIẢI CÂU 4 (PRODUCER - CONSUMER SEMAPHORES)

1. Khai báo & Khởi tạo Semaphore:

```
semaphore mutex = 1; // Độc quyền truy xuất Buffer
semaphore empty = N; // Đếm số ô trống (ban đầu N = 5)
semaphore full = 0; // Đếm số ô có dữ liệu (ban đầu 0)
```

2. Mã nguồn Tiến trình Producer:

```
void producer() {
    while (true) {
        item = produce_item();
```

```
wait(empty); // Chờ có ô trống (empty > 0)
wait(mutex); // Khóa Critical Section

insert_item(item); // Đưa item vào Buffer

signal(mutex); // Mở khóa CS
signal(full); // Tăng số ô có dữ liệu
}
}
```

3. Mã nguồn Tiến trình Consumer:

```
void consumer() {
    while (true) {
        wait(full); // Chờ có dữ liệu (full > 0)
        wait(mutex); // Khóa Critical Section

        item = remove_item(); // Lấy item ra khỏi Buffer

        signal(mutex); // Mở khóa CS
        signal(empty); // Tăng số ô trống
        consume_item(item);
    }
}
```

PHẦN 3: LỜI GIẢI CÂU 5 (BỘ NHỚ & ẢNH XẠ ĐỊA CHỈ)

1. a) Tính số bit và số mục Bảng trang:

- Page size $4KB = 2^{12}$ bytes $\Rightarrow$ Số bit offset $d = 12$ bits.
- Tổng số bit địa chỉ ảo = 32 bits $\Rightarrow$ Số bit chỉ số trang $p = 32 - 12 = 20$ bits.
- Số mục trong Bảng trang = $2^{20} = 1,048,576$ trang (≈ 1 triệu mục).

2. b) Tách địa chỉ ảo $V = 16390$ và tính Địa chỉ vật lý:

- Chỉ số trang $p = \lfloor 16390/4096 \rfloor = 4$.
- Offset $d = 16390 \pmod{4096} = 16390 - 4 \times 4096 = 16390 - 16384 = 6$.
- Địa chỉ vật lý khi $f = 5$: $\text{Physical} = f \times \text{Page_Size} + d = 5 \times 4096 + 6 = 20480 + 6 = 20486$.

HOÀN THÀNH CẨM NANG IT007 UIT!

Chúc bạn ôn tập tốt, làm chủ toàn bộ kiến thức và đạt điểm **A/A+** môn Hệ điều hành IT007!

PHỤ LỤC — KỸ NĂNG THỰC HÀNH

Linux Survival Kit cho Môn Học IT007 UIT

Môi trường WSL2 → Bảng lệnh Linux cốt lõi → Mẫu mã thực hành chọn lọc

 Mục Tiêu Phụ Lục

- Thiết lập môi trường làm việc Linux Ubuntu trên Windows bằng WSL2 nhanh chóng.
- Thành thạo các lệnh kiểm tra hệ thống, tiến trình và biên dịch C (`ps`, `top`, `pstree`, `kill`, `gcc`, `gdb`, `strace`).
- Đọc, chạy và điều chỉnh các mẫu mã minh họa cho những chủ đề Lab IT007 tiêu biểu; đối chiếu đề bài, yêu cầu nộp bài và môi trường lớp trước khi dùng.

1. BẢNG CÁC LỆNH LINUX BẮT BUỘC NẮM VỮNG FOR IT007

LỆNH (COMMAND)	CHỨC NĂNG CHÍNH	CÚ PHÁP MẪU THÔNG DỤNG
<code>ps</code>	Liệt kê các tiến trình đang chạy trong hệ thống.	<code>ps aux</code> hoặc <code>ps -ef</code> (xem tất cả PID, PPID, CPU, MEM)
<code>pstree</code>	Hiển thị sơ đồ cây tiến trình (Process Tree) trực quan.	<code>pstree -p</code> (hiển thị kèm PID)
<code>top</code> / <code>htop</code>	Theo dõi thời gian thực mức tiêu thụ CPU, RAM của các tiến trình.	<code>top</code> (nhấn `q` để thoát)
<code>kill</code>	Gửi tín hiệu Signal đến tiến trình để dừng/hủy tiến trình.	<code>kill -9 <PID></code> (gửi SIGKILL ép hủy ngay)
<code>gcc</code>	Trình biên dịch mã nguồn C trên Linux.	<code>gcc -Wall program.c -o program -lrt -lpthread</code>
<code>strace</code>	Theo dõi tất cả các System Calls mà chương trình thực thi.	<code>strace ./program</code> (cực hay để debug System Call)

2. MẪU MÃ MINH HỌA CHO MỘT SỐ CHỦ ĐỀ LAB IT007

Các đoạn dưới đây là tài liệu học tập chọn lọc, không phải bộ mã nguồn hoàn chỉnh hoặc đề nộp bài cho toàn bộ Lab 1–6. Chỉ những đoạn được ghi rõ là chương trình hoàn chỉnh mới nên được biên dịch trực tiếp.

Ví dụ Bash: kiểm tra số và tính tổng (`check\_number.sh`)

```
#!/bin/bash
# lab2_script.sh - Kiểm tra số nguyên tố và tính tổng
read -p "Nhập số nguyên N: " N

if [ $N -le 0 ]; then
    echo "Vui lòng nhập số nguyên dương!"
    exit 1
fi
```

```
sum=0
for (( i=1; i<=N; i++ )); do
    sum=$((sum + i))
done

echo "Tổng từ 1 đến $N là: $sum"
```

Ví dụ POSIX: tiến trình và bộ nhớ chia sẻ (Shared Memory)

```
/* lab3_shm.c - Sử dụng POSIX Shared Memory shm_open & mmap có kiểm tra lỗi */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    const int SIZE = 4096;
    const char *name = "IT007_Shm";

    int fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    if (fd < 0) { perror("shm_open failed"); exit(1); }
    ftruncate(fd, SIZE);

    void *ptr = mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (ptr == MAP_FAILED) { perror("mmap failed"); exit(1); }

    pid_t pid = fork();
    if (pid < 0) { perror("fork failed"); exit(1); }

    if (pid == 0) { // Child Writes
        sprintf((char*)ptr, "Hello from Child Process! PID = %d", getpid());
        exit(0);
    } else { // Parent Reads
        wait(NULL);
        printf("Parent read from Shared Memory: %s\n", (char*)ptr);
        munmap(ptr, SIZE);
        shm_unlink(name);
    }

    return 0;
}
```

Ví dụ cấu trúc dữ liệu: mô phỏng định thời FCFS

```
/* lab4_fcfs.c - Struct dữ liệu tính WT, TAT định thời */
typedef struct {
    int id, at, bt, ct, tat, wt;
} Process;

void calculate_fcfs(Process p[], int n) {
    int current_time = 0;
    for (int i = 0; i < n; i++) {
        if (current_time < p[i].at) current_time = p[i].at;
        p[i].ct = current_time + p[i].bt;
        p[i].tat = p[i].ct - p[i].at;
    }
}
```

```

        p[i].wt = p[i].tat - p[i].bt;
        current_time = p[i].ct;
    }
}

```

Ví dụ POSIX Threads với semaphore (`pthread\_sem.c`)

```

/* lab5_pthread.c - Độc quyền truy xuất Mutex bằng POSIX Semaphore */
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t mutex;
int counter = 0;

void* worker(void* arg) {
    sem_wait(&mutex);
    counter++;
    printf("Thread %ld incremented counter to %d\n", (long)arg, counter);
    sem_post(&mutex);
    return NULL;
}

int main() {
    pthread_t threads[5];
    sem_init(&mutex, 0, 1);
    for (long i = 0; i < 5; i++) pthread_create(&threads[i], NULL, worker, (void*)i);
    for (int i = 0; i < 5; i++) pthread_join(threads[i], NULL);
    sem_destroy(&mutex);
    return 0;
}

```

Ví dụ chương trình hoàn chỉnh: pipe với `fork()` và `execvp()` (`lab6\_pipe.c`)

```

/* lab6_pipe.c - Chương trình biên dịch hoàn chỉnh thực thi lệnh: ls -l | grep txt */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char *cmd1[] = {"ls", "-l", NULL};
    char *cmd2[] = {"grep", "txt", NULL};

    int pipefd[2];
    if (pipe(pipefd) < 0) { perror("pipe failed"); exit(1); }

    pid_t p1 = fork();
    if (p1 < 0) { perror("fork 1 failed"); exit(1); }

    if (p1 == 0) { // Child 1: Chạy 'ls -l' và ghi vào pipe
        close(pipefd[0]); // Đóng đầu đọc không dùng
        if (dup2(pipefd[1], STDOUT_FILENO) < 0) { perror("dup2 failed"); exit(1); }
        close(pipefd[1]);
        execvp(cmd1[0], cmd1);
        perror("execvp cmd1 failed");
        exit(1);
    }
}

```



```
pid_t p2 = fork();
if (p2 < 0) { perror("fork 2 failed"); exit(1); }

if (p2 == 0) { // Child 2: Chạy 'grep txt' và đọc từ pipe
    close(pipefd[1]); // Đóng đầu ghi không dùng
    if (dup2(pipefd[0], STDIN_FILENO) < 0) { perror("dup2 failed"); exit(1); }
    close(pipefd[0]);
    execvp(cmd2[0], cmd2);
    perror("execvp cmd2 failed");
    exit(1);
}

// Tiến trình cha đóng cả 2 đầu pipe và chờ 2 con hoàn thành
close(pipefd[0]);
close(pipefd[1]);
waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);
return 0;
}
```

🚩 Lời khuyên đạt 10/10 Thực Hành Lab IT007

- Luôn kiểm tra mã lỗi trả về của tất cả System Calls (`if (pid < 0)`, `if (fd < 0)`).
- Dọn dẹp tài nguyên (dùng `wait()`, `munmap()`, `shm\_unlink()`, `close()`) để tránh rò rỉ RAM và Zombie process.